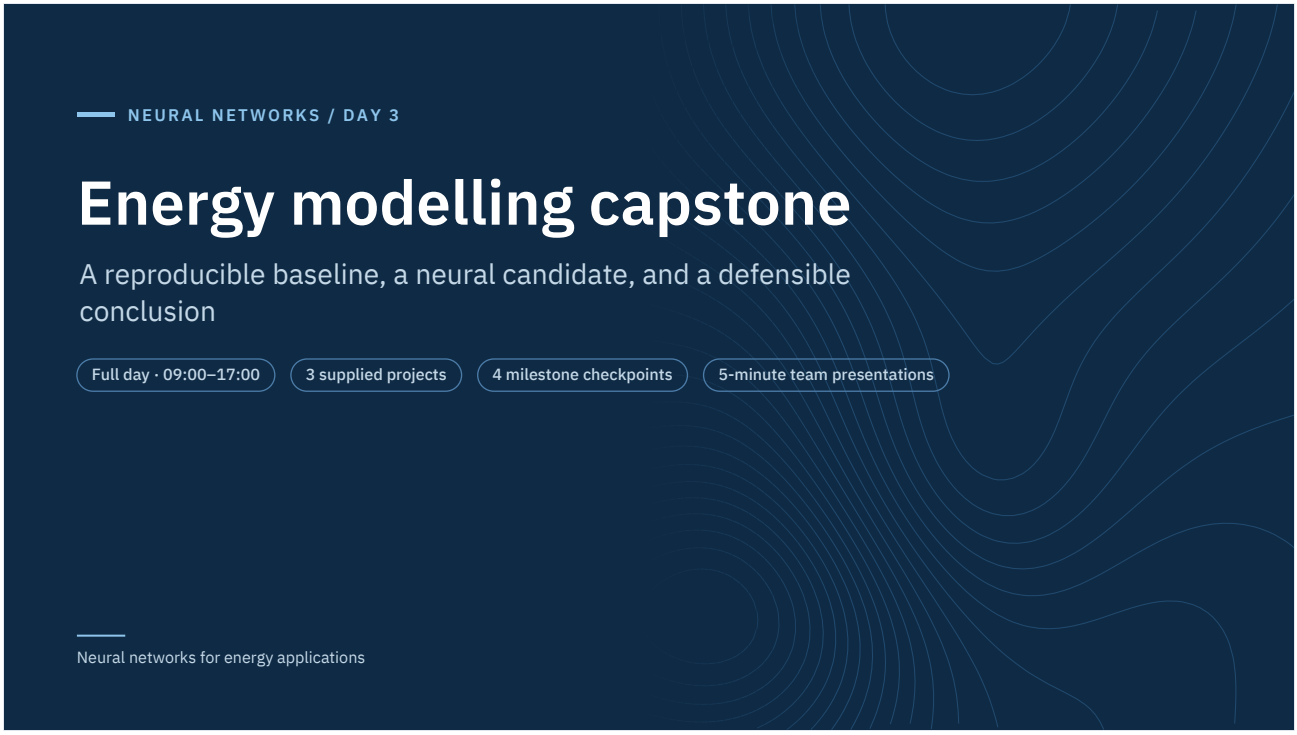
Energy modelling capstone



This midpoint capstone uses the dense-network and training skills from Days 1 and 2. Teams of two or three choose one of three supplied projects: ESP gas lock (classification), PE log-curve prediction (regression) or anonymous sensor signals (classification). CNNs and sequence models are optional follow-on work after Days 4 and 5, not prerequisites for today's assessment.

Set expectations in the first minute. Most of the day is project work in five sprints. After the morning briefings every lecture is short (15-20 minutes) and prepares the next sprint. Four milestone checkpoints gate progress: data contract, baseline, controlled experiment, locked final assessment. The day ends with five-minute team presentations scored with the rubric shown later.

Say explicitly that the goal is sound evidence, not a leaderboard. A network that does not beat the baseline, reported honestly, can earn full marks. [1 min]

Today's plan



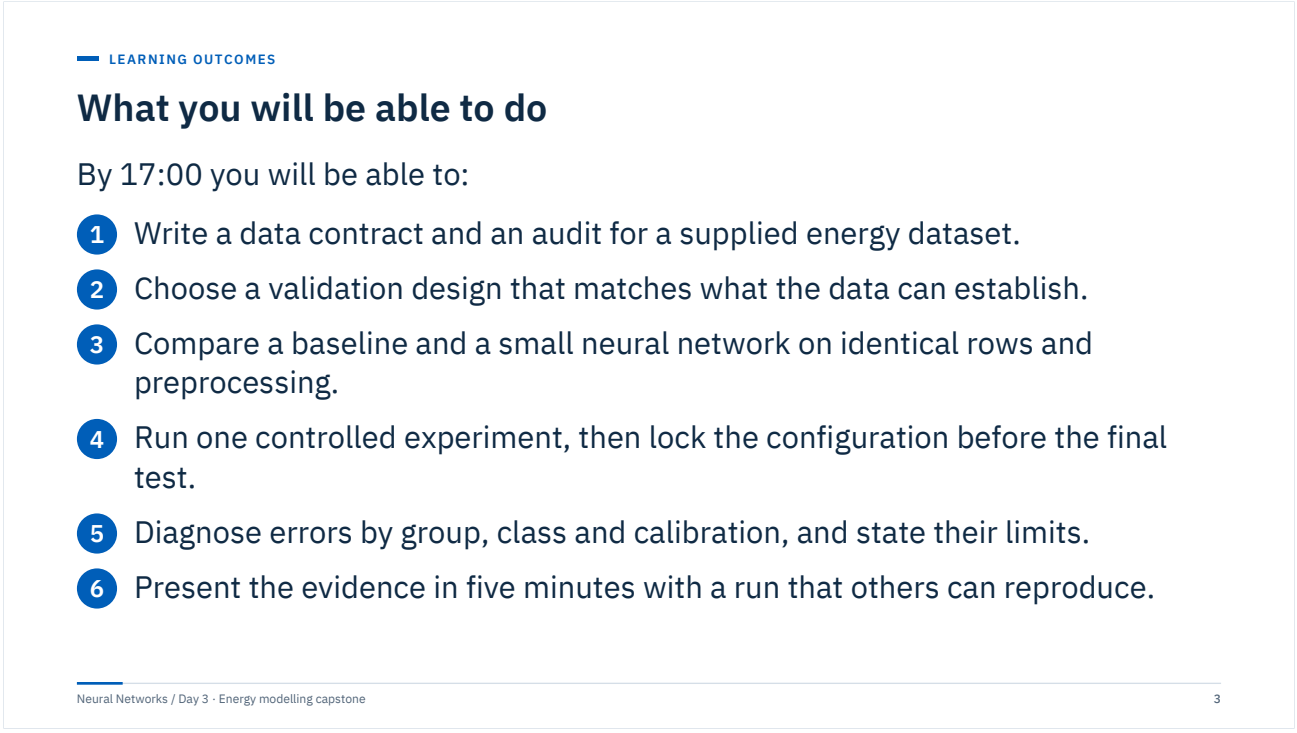
Walk through the rhythm rather than every row: the brief and three project briefings fill the morning until 10:45, then sprints, checkpoints and short lectures alternate. Checkpoints are ten minutes, so each team gets about two minutes at its own screen; circulate in a fixed order and keep a written log of what each team showed.

The day adds up to 165 minutes of lectures (including the close), 170 minutes of sprints, four 10-minute checkpoints, 35 minutes of presentations and 70 minutes of breaks.

Timing levers if the class runs late: shorten the project briefings (teams revisit their own project's slides during Sprint 1), or trim Sprint 5. Do not cut the checkpoints or the evaluation work in Sprints 3 and 4; the instructor guide says not to remove the evaluation debriefs to recover time.

Presentations take 35 minutes: five teams at seven minutes each (five minutes plus two minutes of questions). With six teams, trim questions to one minute and take the extra minute from the close. [2 min]

What you will be able to do



Each outcome maps to evidence you will see during the day. Outcome 1 is checkpoint 1 (the data contract). Outcome 2 is the split policy in that contract. Outcome 3 is checkpoint 2 (the score table with a reference and a baseline). Outcome 4 is checkpoint 3 (hypothesis, several seeds, locked configuration). Outcome 5 is the error-analysis figure at checkpoint 4. Outcome 6 is the presentation and the handoff.

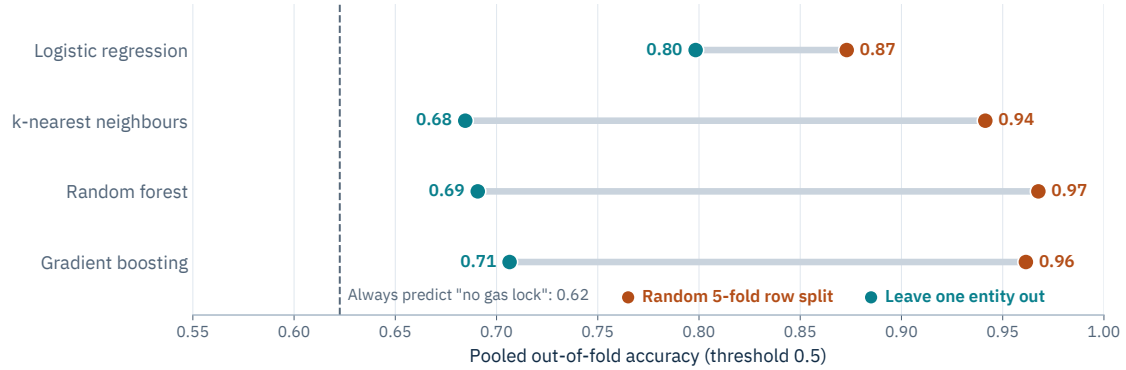
Ask the room which outcome they expect to be hardest. Most groups say "beat the baseline"; the honest answer is usually outcome 2, because every supplied file has a structure (entities, depth blocks, near-duplicate records) that a random split would hide. The next slide shows how much that matters. [2 min]

Random row splits flattered every ESP model

MOTIVATING CASE

Random row splits flattered every ESP model

Same file, same four models, two validation designs. Which number answers "will it work on the next pump"?



Computed from esp_train.csv (3,243 rows, 7 entities): pooled out-of-fold accuracy at threshold 0.5, preprocessing fitted inside each fold.

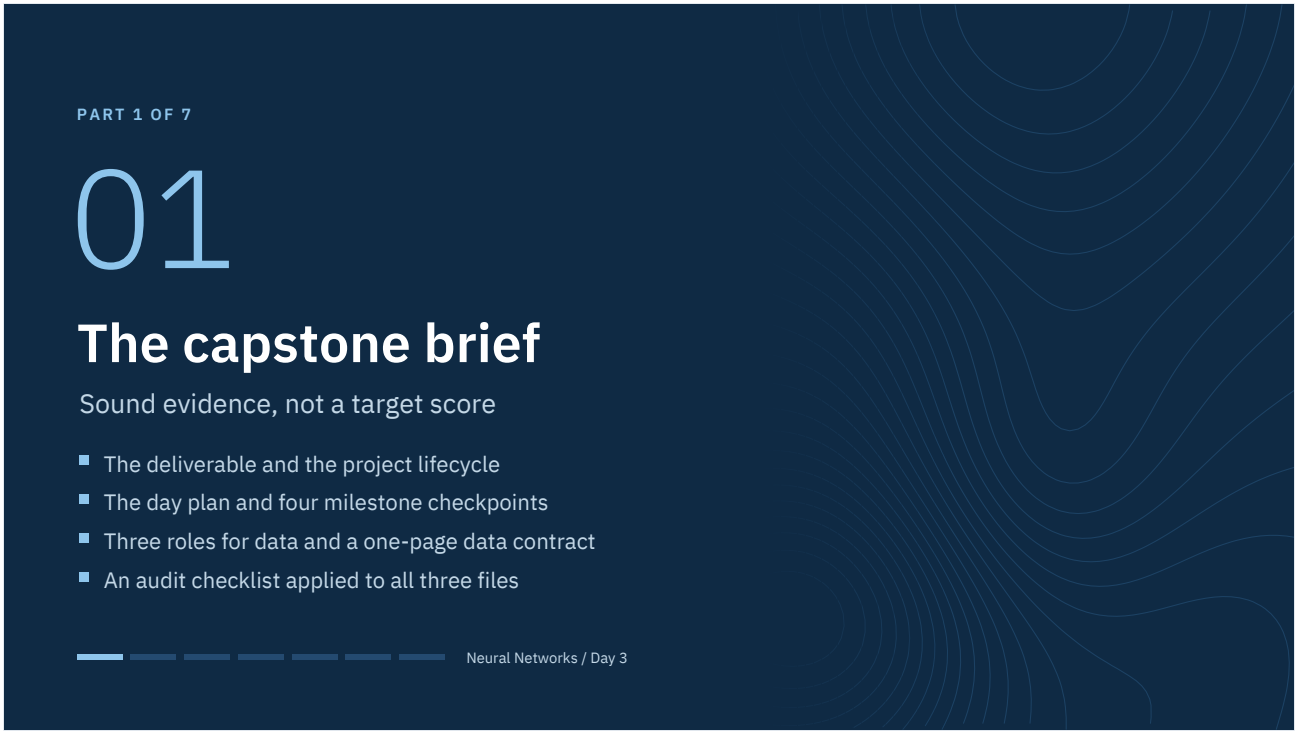
Neural Networks / Day 3 · Energy modelling capstone 4

This is the motivating case for the whole day. The orange dots come from a random five-fold row split: rows from the same pump, one hour apart, land on both sides, so the model is scored on near-copies of its training rows. The teal dots come from leave-one-entity-out: each pump is scored by a model that never saw it. That is the deployment question for a new ESP.

Read the numbers. Under the entity holdout, k-nearest neighbours (0.68), random forest (0.69) and gradient boosting (0.71) barely beat the majority rule "always no gas lock" (0.62). Logistic regression drops least, from 0.87 to 0.80. The ranking of models flips: the most flexible models, which memorize set-points of individual pumps, lose most. Average precision tells the same story (random 0.92-0.98, entity holdout 0.56-0.87).

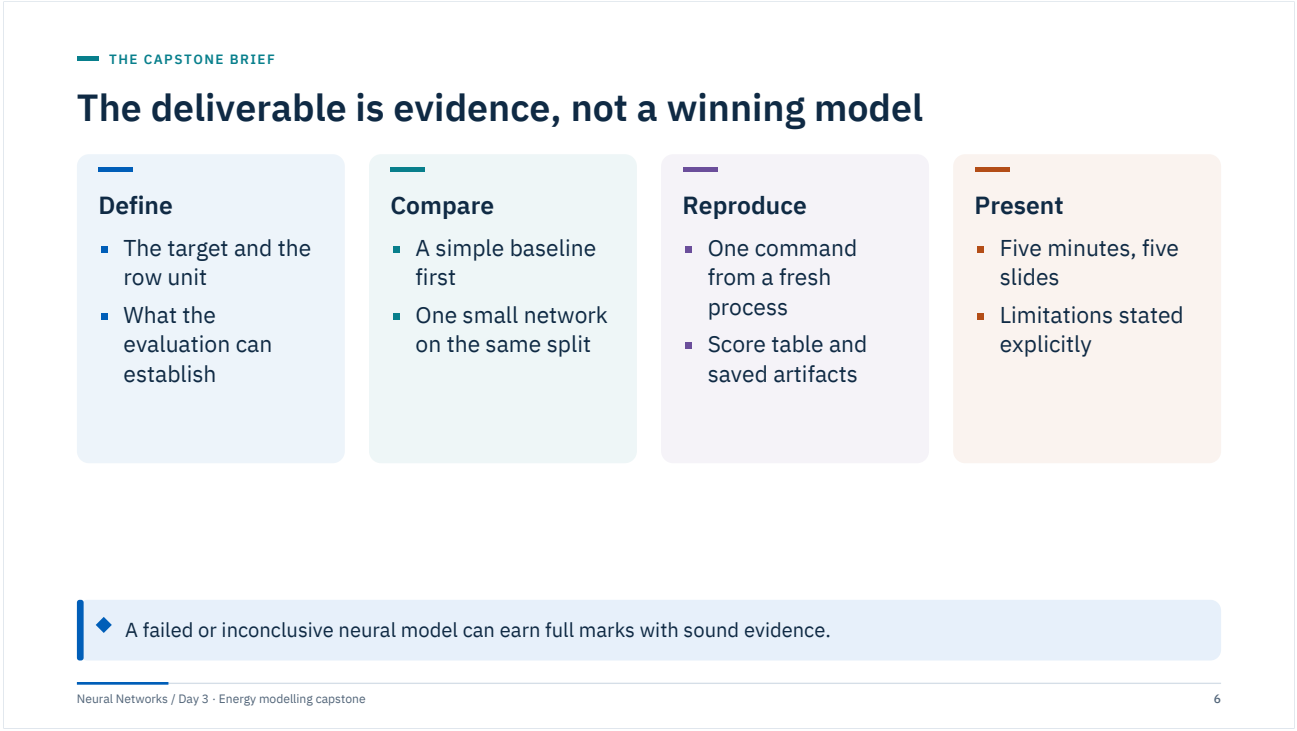
Ask: which number would you put in a report? The teal ones, and even those hide large differences between pumps, as we will see in error analysis. Transition: the capstone brief. [4 min]

The capstone brief



This part sets the rules of the day in about 25 minutes. Everything here applies to all three projects; the project-specific details follow in parts 2-4. Ask teams to sit together by the end of this part; the project choice itself happens after the three briefings, at the start of Sprint 1. Keep an eye on the clock: the ESP briefing starts at 09:35 and Sprint 1 at 10:45. [0.5 min]

The deliverable is evidence, not a winning model

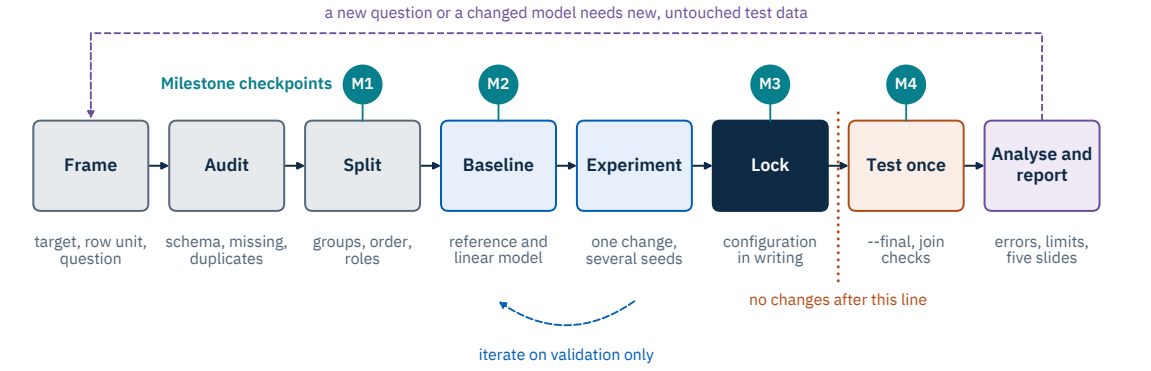


The goal is sound experimental work rather than a target score. Remove any expectation that a neural model must win: on these files it often does not, and a clear negative result is a valid finding. Leakage or incorrect target alignment, on the other hand, invalidates the affected assessment regardless of the score.

Assign teams of two or three. Suggest roles that rotate: one person drives the notebook, one audits data and checks invariants, one keeps the evidence log (commands, seeds, numbers, decisions) that becomes the presentation. All members share the final reasoning and must be able to answer questions about any slide.

Ask: what would make a "95% accurate" result worthless here? Expected answers: the wrong split, test labels used for tuning, a label join that silently misaligned rows. [2 min]

Every project follows the same eight-stage lifecycle



Walk left to right. Frame: the target, the row unit and the question. Audit: schema, missing values, duplicates, keys, order. Split: decide which rows play which role, respecting groups and order. Baseline: a reference and a linear model on that split. Experiment: one controlled change over several seeds. Lock: write the configuration down. Test once: run the locked configuration on the supplied test file and check the joins. Analyse and report: error analysis, limits, five slides.

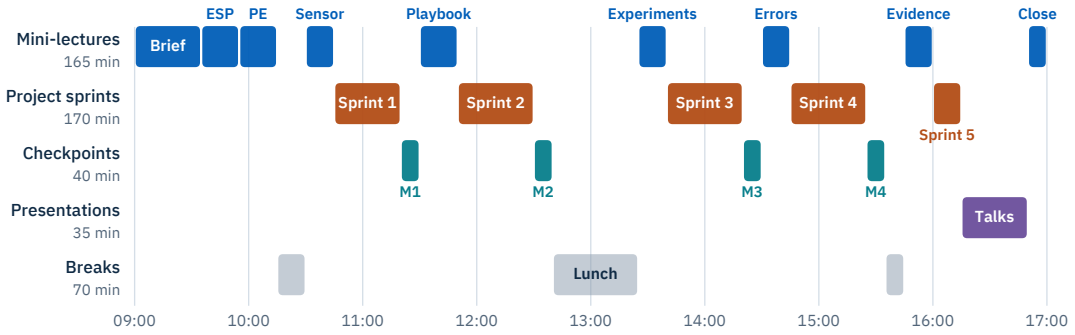
Two loops matter. The blue loop (baseline and experiment) may repeat as often as you like, because it only uses validation data. The purple loop (a new question or a changed model) needs new, untouched test data; you cannot loop back after the dotted line and keep calling the test "final".

The checkpoints sit where mistakes are cheapest to fix: after the split (M1), after the baseline (M2), at the lock (M3) and after the single test run (M4). [3 min]

Short lectures alternate with five project sprints

THE CAPSTONE BRIEF

Short lectures alternate with five project sprints



Schematic of today's plan: 165 min of lectures including the close, 170 min of sprints, four 10-min checkpoints, 35 min of presentations and 70 min of breaks.

Neural Networks / Day 3 · Energy modelling capstone 8

Show the rhythm: every mini-lecture is followed by the sprint that uses it. The playbook lecture feeds Sprint 2, the experiment-design lecture feeds Sprint 3, error analysis feeds Sprint 4 and the evidence lecture feeds Sprint 5 and the presentations. The morning is lecture-heavy because teams need all three project briefings before they commit; after 10:45 no lecture is longer than 20 minutes.

During sprints, circulate and ask the checkpoint questions early so that nobody is surprised at the gate. Useful prompts: "which rows are in your validation set?", "where were the imputer medians computed?", "what would change your conclusion?".

Checkpoints are short by design. A team that is not ready at a checkpoint keeps working, and you revisit it at the start of the next sprint; do not let one team hold up the room. If more than half the teams miss a gate, extend that sprint by ten minutes and shorten Sprint 5 instead. [2 min]

Each team chooses one of three supplied projects

THE CAPSTONE BRIEF

Each team chooses one of three supplied projects

Project	Actual target	Supplied training data	Validation fallback
ESP gas lock	GAS_LOCK, values 0 and 1	3,243 rows, 7 entities	Hold out 2 entities
PE log curve	PE, numeric	2,922 rows, 6 input columns	Last 20% of source rows
Sensor signals	target, values −1 and 1	3,580 rows, 10 inputs plus uuid	Stratified 20% of rows

Counts describe the supplied course files.

◆ Each fallback split has a stated limit; a team may propose a better split at checkpoint 1.

Neural Networks / Day 3 · Energy modelling capstone 9

These counts come from the provided CSVs, not external benchmarks. The log project predicts photoelectric-factor values labelled PE; the supplied files do not contain a shear-sonic DTS target. Sensor columns lack documented physical semantics, so do not invent units or a fault interpretation.

The last column is the split used by the reference workflow in curriculum/capstone.py. Each is a documented fallback with a stated limit: the ESP holdout tests unseen entities, not future time; the PE block cannot establish cross-well performance; the sensor split cannot establish independence. Teams may propose a better split at checkpoint 1 if they can justify it.

Let teams choose by interest, but balance the room: aim for at least one team per project so the presentations compare different kinds of evidence. The three notebooks are in exercises/01_esp_gas_lock, exercises/02_log_curves and exercises/03_sensor_signals. [2 min]

Four milestones gate the day

THE CAPSTONE BRIEF

Four milestones gate the day

Milestone	Due	Evidence to show
M1 Data contract	11:20	Schema, target, audit findings, split proposal
M2 Baseline	12:30	Reproducible validation result and learning curve
M3 Controlled experiment	14:20	Hypothesis, seed spread, locked configuration
M4 Locked final assessment	15:25	Final scores, join checks, one error plot
Presentations	16:15	Five slides, one command, predictions with ids

◆ A team that misses a gate keeps working on it; the gate moves, the evaluation standard does not.

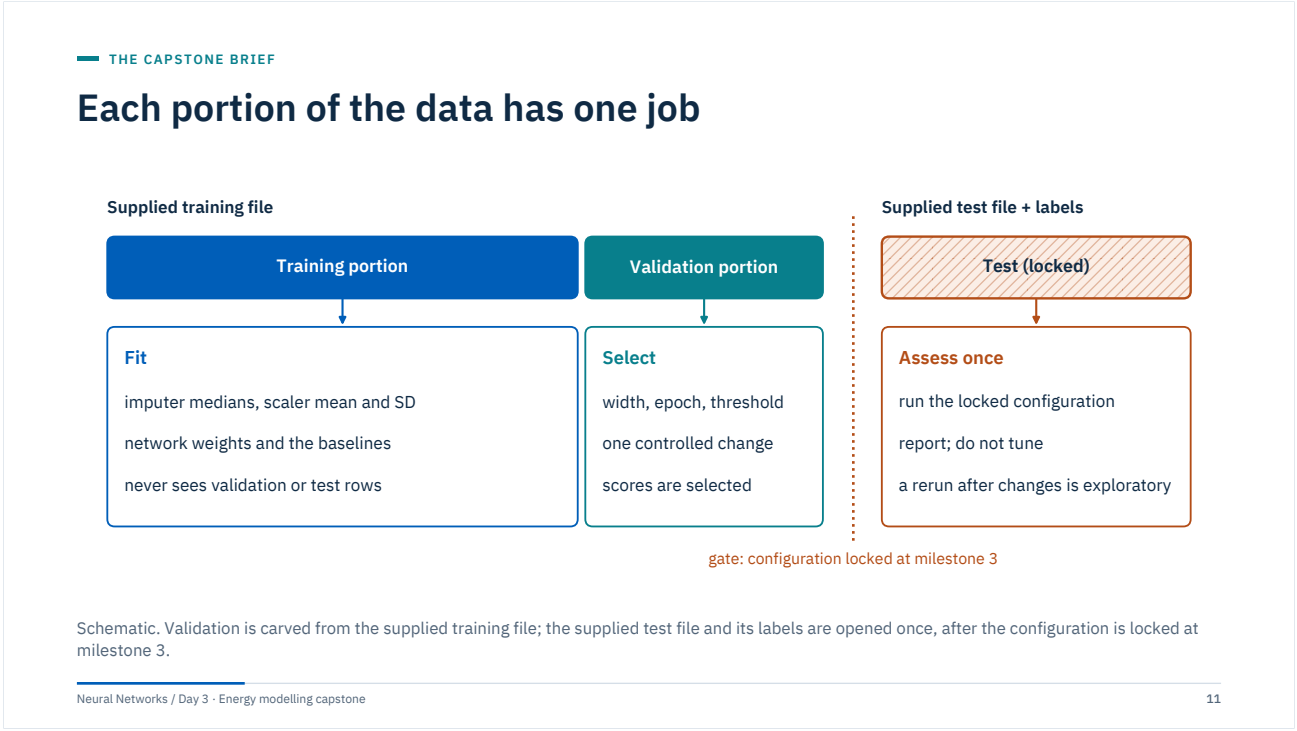
Neural Networks / Day 3 · Energy modelling capstone 10

At each gate, review evidence before teams proceed. If the baseline is incomplete, defer architecture expansion rather than shortening evaluation: a team with a solid baseline and no network has a better presentation than a team with a network and no trustworthy split.

The five sprints total 170 minutes: 35 for the contract and audit, 40 each for the baselines, the experiment and the final assessment, and 15 for the handoff.

Adapt presentation time to the number of teams. Write the four due times on the board and keep them visible all day. [1.5 min]

Each portion of the data has one job

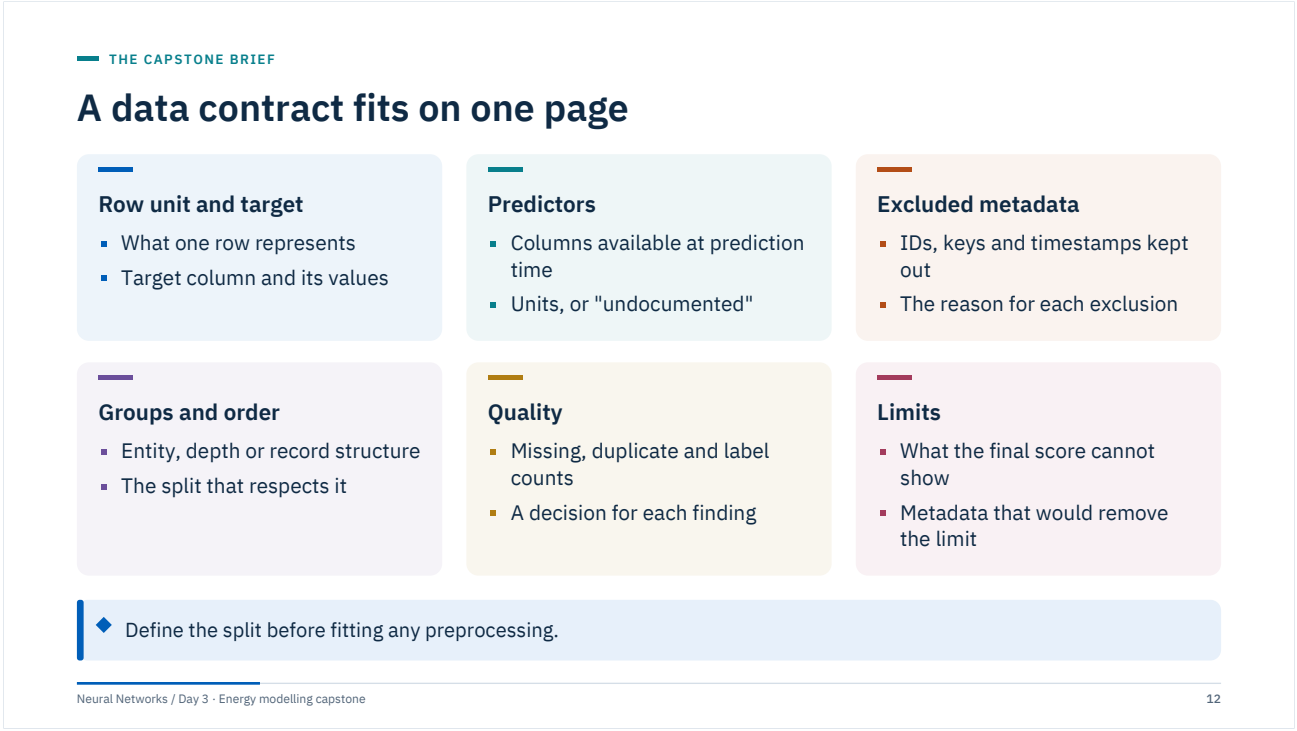


Three roles, three rules. The training portion of the supplied train file fits everything that is fitted: imputer medians, scaler means and standard deviations, network weights and the baselines. The validation portion, also carved from the train file, selects: width, epoch, threshold, features. Scores on it are selected estimates, so they are optimistic. The supplied test file and its labels assess the locked configuration once.

How each project carves validation: ESP holds out two whole entities (esp_30 and esp_31 with seed 42); PE holds out the last 20% of source rows; sensor holds out a stratified 20% of rows.

For the log target file, alignment is positional because no key is supplied. Preserve the test row order and verify row counts. For ESP and sensor targets, use uuid joins with one-to-one validation rather than assuming row order. [3 min]

A data contract fits on one page



Require a brief written data contract before any model is fitted; it is the evidence for checkpoint 1. A numeric ID is not automatically a predictor: ENTITY, uuid and TimeSlot are metadata unless the team argues otherwise, and an argument for using them must survive the deployment question.

A missing data dictionary limits physical claims even when a model can be fitted successfully. Write "units undocumented" rather than guessing psi, bbl/d or API units from column names.

Keep supplied label files out of model selection. The contract should name them and say when they will be opened (in Sprint 4, after the lock at milestone 3).

A good ESP limit statement: "The supplied test contains three unseen entities over the same dates, so it can show unseen-entity classification of current states, not advance warning." [3 min]

The audit checklist flags issues in every file

THE CAPSTONE BRIEF

The audit checklist flags issues in every file

	ESP gas lock	PE log curve	Sensor signals
Shape and schema	✓ 3,243 rows × 9 columns	✓ 2,922 rows × 7 columns	✓ 3,580 rows × 12 columns
Target and balance	✓ GAS_LOCK = 1 in 37.7% of rows	✓ PE 0.20–8.09, median 3.51	! labels −1/1 (57.6% = 1): map to 0/1
Missing values	! 300 cells, all in 50 esp_39 rows	✓ none	✓ none
Duplicates	! 2 identical rows; 6 repeat stamps	! 15 rows; a 12-row block repeats	✗ 2,484 rows repeat a feature vector
Keys and joins	✓ uuid unique in test; join on uuid	! no key: labels align by position	✓ uuid unique; labels join on uuid
Groups and order	✓ 7 entities, hourly TimeSlot	✗ no well ID; 7 depth resets	! no groups; uuid order has long runs
Units and meaning	! units undocumented	! units undocumented	! meaning and units undocumented
Test file inputs	✓ 3 unseen entities, same dates	✓ 310 rows, Depth 2964.5–3122.5	✗ 646 of 895 repeat a training row

Computed from the supplied training files and the test input files; no test labels were read. Tick: fine. Gold: needs a documented decision. Orange cross: limits what the evaluation can show.

Neural Networks / Day 3 · Energy modelling capstone 13

The first column is the checklist teams apply in Sprint 1: shape and schema, target and balance, missing values, duplicates, keys and joins, groups and order, units and meaning, and the test file's inputs (never its labels).

Highlights to say aloud. ESP: all 300 missing cells sit in 50 rows of esp_39 where all six sensors are missing at once; two entities also have repeated or out-of-order timestamps. PE: there is no well identifier, the file has seven depth resets, and a 12-row block (rows 2,297-2,308) repeats rows 450-461 exactly. Sensor: 2,484 of 3,580 rows repeat another row's ten values exactly, and 646 of the 895 test rows repeat a training row, so the supplied test largely re-scores records the model has seen. The duplicate counts on the board: ESP's two identical rows are repeated all-missing esp_39 rows stamped 20 January 10:00; PE's 15 duplicated rows are the 12-row block plus the three zero-depth steps, where a row repeats its predecessor.

None of these is a reason to give up. Each needs a documented decision and a sentence in the limits card. [3 min]

Which data may each step use?

PRACTICE

4 min Pairs

Which data may each step use?

For each step, answer: training portion, validation portion, test file, or none.

1. Computing the medians used for imputation
2. Choosing the epoch at which training stops
3. Picking the decision threshold for gas-lock alerts
4. Confirming that test uuids match the label file one-to-one
5. Choosing width 32 over 128 after seeing both test scores

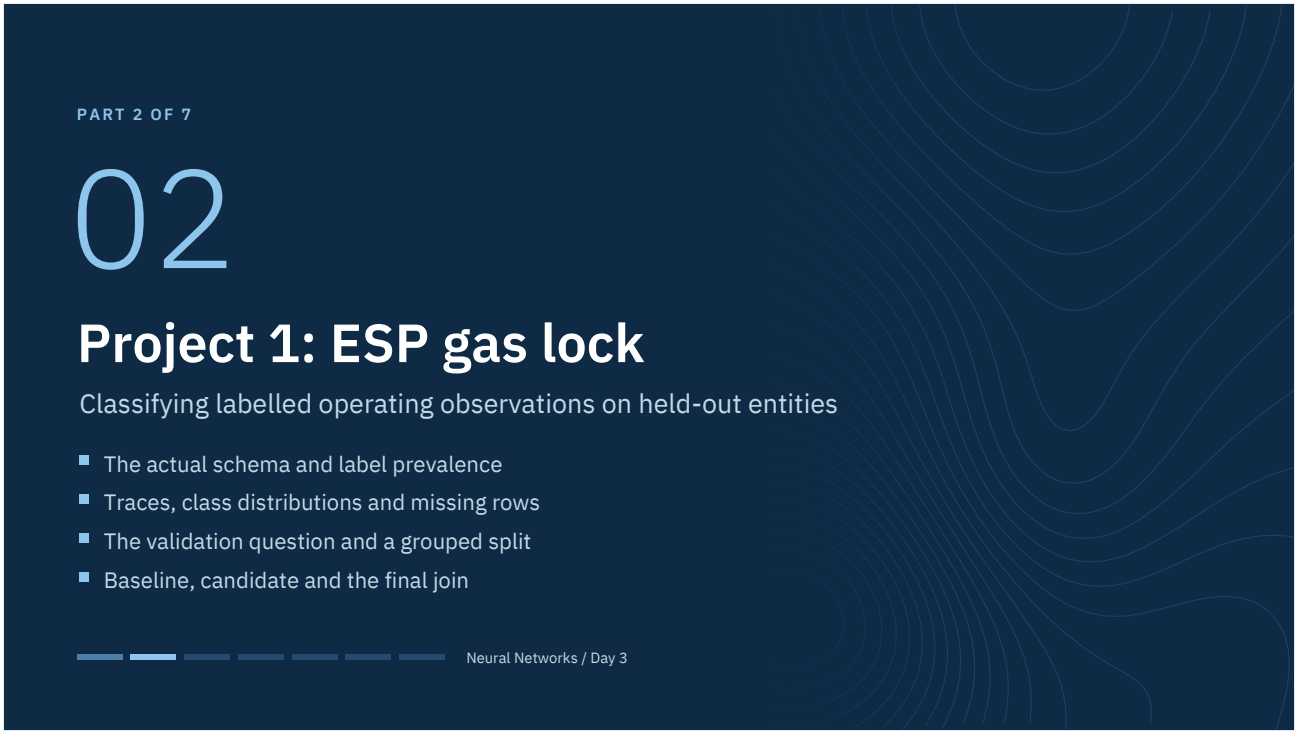
Neural Networks / Day 3 · Energy modelling capstone 14

Answers.

1. Training portion only. Medians are fitted statistics; computing them on all rows lets validation values influence the preprocessing. The reference workflow fits SimpleImputer and StandardScaler on the training rows and only transforms validation and test rows. 2. Validation portion. Early stopping is model selection; the reference loop copies the state with the lowest validation loss. 3. Validation portion. The threshold is a selected parameter. Choosing it on the test file turns the test into a second validation set. The reference workflow chooses it on validation (for example 0.775 for the ESP baseline in the reference run). 4. The test file, but only as an integrity check at milestone 4: merge on uuid with validate="one_to_one" and assert the row count. The check chooses nothing, so it does not bias the assessment. 5. None. Choosing after seeing test scores makes the test a validation set. If it happened, report the test result as exploratory and say that a fresh test set would be needed.

Common error: answering "training" for question 3 because the threshold "is part of the model". It is part of the deployed decision rule, but it is chosen, not fitted. [4 min + 2 min debrief]

Project 1: ESP gas lock



The provided train and test sets contain different ESP entity IDs over overlapping time periods. The supplied test therefore supports an unseen-entity assessment more directly than a future-time assessment. Keep this sentence in mind for every ESP slide: the task is to classify the current state of a pump the model has not seen, not to warn in advance.

This briefing has 20 minutes (09:35-09:55). Keep the code slides brief; ESP teams study them in Sprint 1. [0.5 min]

Six sensors, an entity and a timestamp per row

PROJECT 1: ESP GAS LOCK

Six sensors, an entity and a timestamp per row

Role	Columns	Treatment
Numeric measurements	PIP, PDP, Q, THP, Mamps, PIT	Check missingness and units
Asset grouping	ENTITY	Hold entities together in validation
Ordering	TimeSlot	Parse timestamps, inspect chronology
Target / test key	GAS_LOCK / uuid	Target in train, join key in test

No vibration channels are present in the supplied files.

◆ ENTITY and TimeSlot define the split and the order; they are not model inputs.

Neural Networks / Day 3 · Energy modelling capstone 16

Names suggest pump intake and discharge pressure, flow rate, tubing-head pressure, motor current and intake temperature, but the CSVs alone do not establish every physical unit. Confirm units with the data owner before making engineering threshold claims.

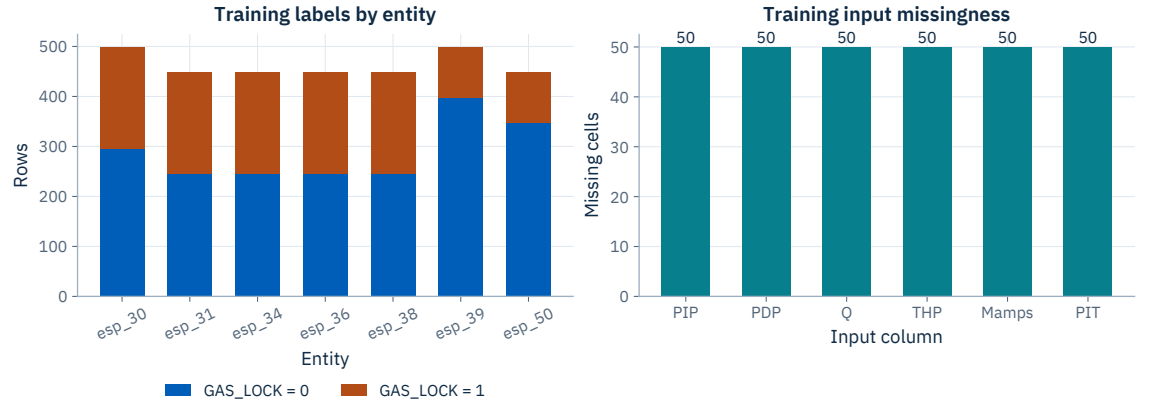
The training file has nine columns and no uuid; the test input file has uuid plus the same columns without GAS_LOCK, and the label file has uuid and GAS_LOCK. The training file has 300 missing cells, all in esp_39.

Ask: which of these columns would be available at prediction time for a new pump? The six measurements and the timestamp. ENTITY is available too, but a model that uses it cannot say anything about a new entity, which is the whole point of the evaluation. [1.5 min]

Label prevalence is measured, not assumed

PROJECT 1: ESP GAS LOCK

Label prevalence is measured, not assumed



Supplied esp_train.csv. Counts are per row, not independent failure events. Missing sensor values require an explicit training-only treatment.

Neural Networks / Day 3 · Energy modelling capstone 17

The training labels contain 2,019 zeros and 1,224 ones, or about 37.7% positive rows. This teaching dataset is not an extremely rare-event dataset at row level. Per-entity differences matter more than a generic prevalence assumption: five entities have 204 labelled rows each, while esp_39 and esp_50 have 102 each, so their positive rates are about 20% and 23%.

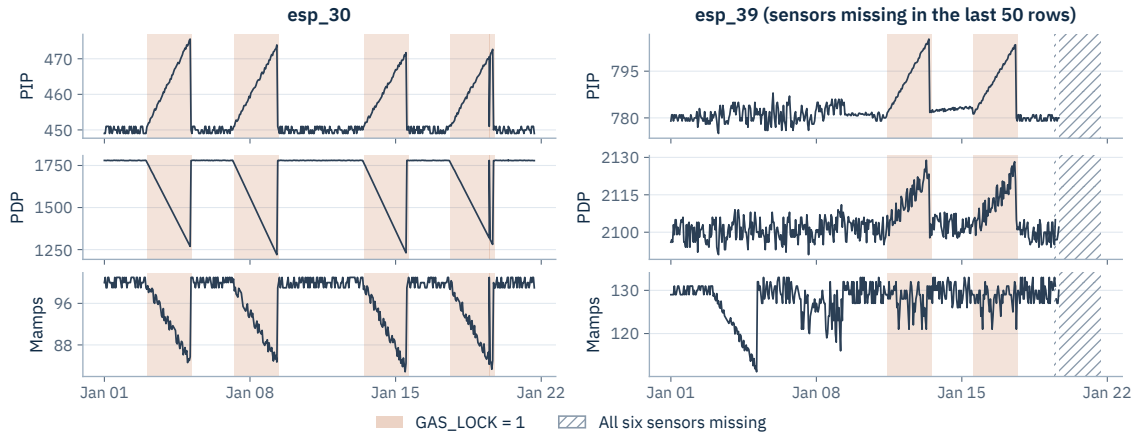
Counts are rows, not events. Each labelled period is a block of consecutive hourly rows, so 1,224 positive rows correspond to only 24 labelled periods (four in each of five entities, two in each of the other two). Event-level metrics, covered in ML Module 5, would count those periods, not the rows.

The right panel shows 50 missing cells per sensor; the next slides show that they all come from the same 50 rows of one entity. [1.5 min]

Gas-lock periods show up as ramps in the traces

PROJECT 1: ESP GAS LOCK

Gas-lock periods show up as ramps in the traces



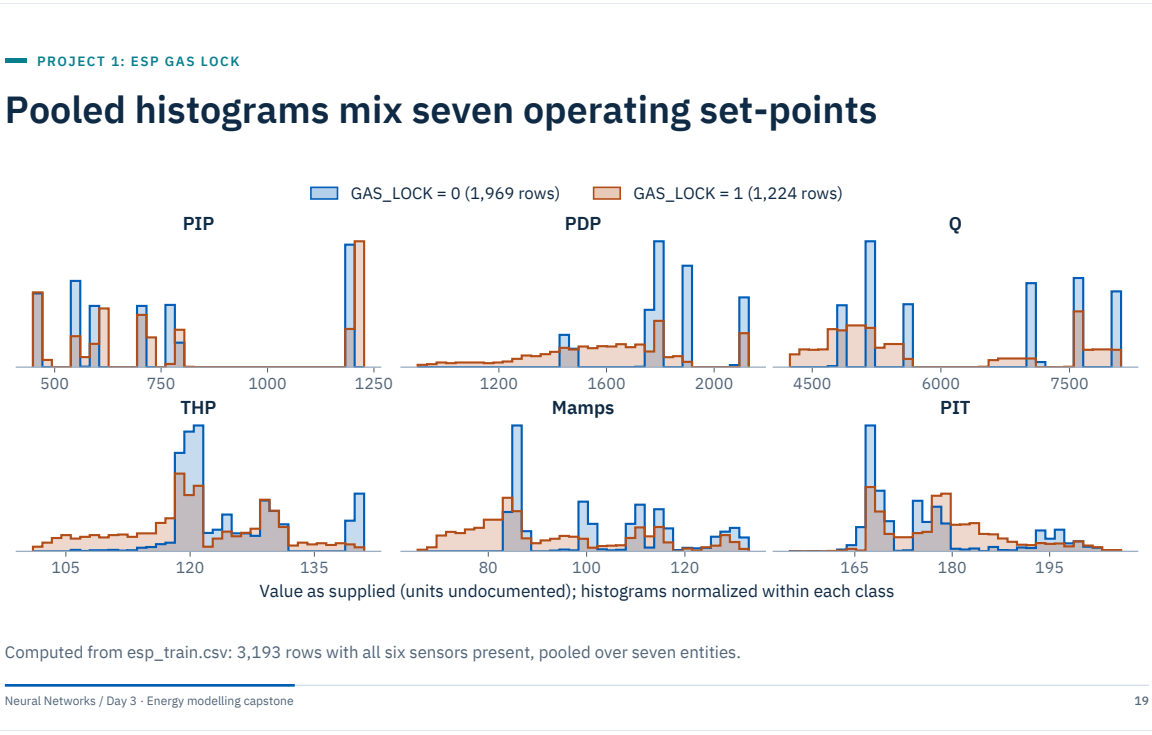
Computed from esp_train.csv: entities esp_30 and esp_39 in TimeSlot order; values as supplied, units undocumented.

In esp_30, every labelled period looks the same: pump intake pressure (PIP) ramps up, discharge pressure (PDP) and motor current (Mamps) ramp down, then all three snap back to their set-points. esp_39 behaves differently: PDP rises slightly during its labelled periods, Mamps ramps down on 3-5 January without any label (the dates of the first labelled period in all six other pumps; possible label noise, a question for the data owner), and the last 50 rows have all six sensors missing.

Two honest observations for the data contract. First, every labelled period in the file lasts exactly 51 hourly rows and starts a multiple of 50 rows after the entity's first row; that regularity suggests curated or simulated labels, so ask the data owner how labels were produced before drawing physical conclusions. Second, the labels describe the current state; nothing here measures how early a gas lock could be detected.

Ask: would a model trained on pumps like esp_30 recognize esp_39's labelled periods? Hold the answer; the error-analysis part shows it. [2.5 min]

Pooled histograms mix seven operating set-points

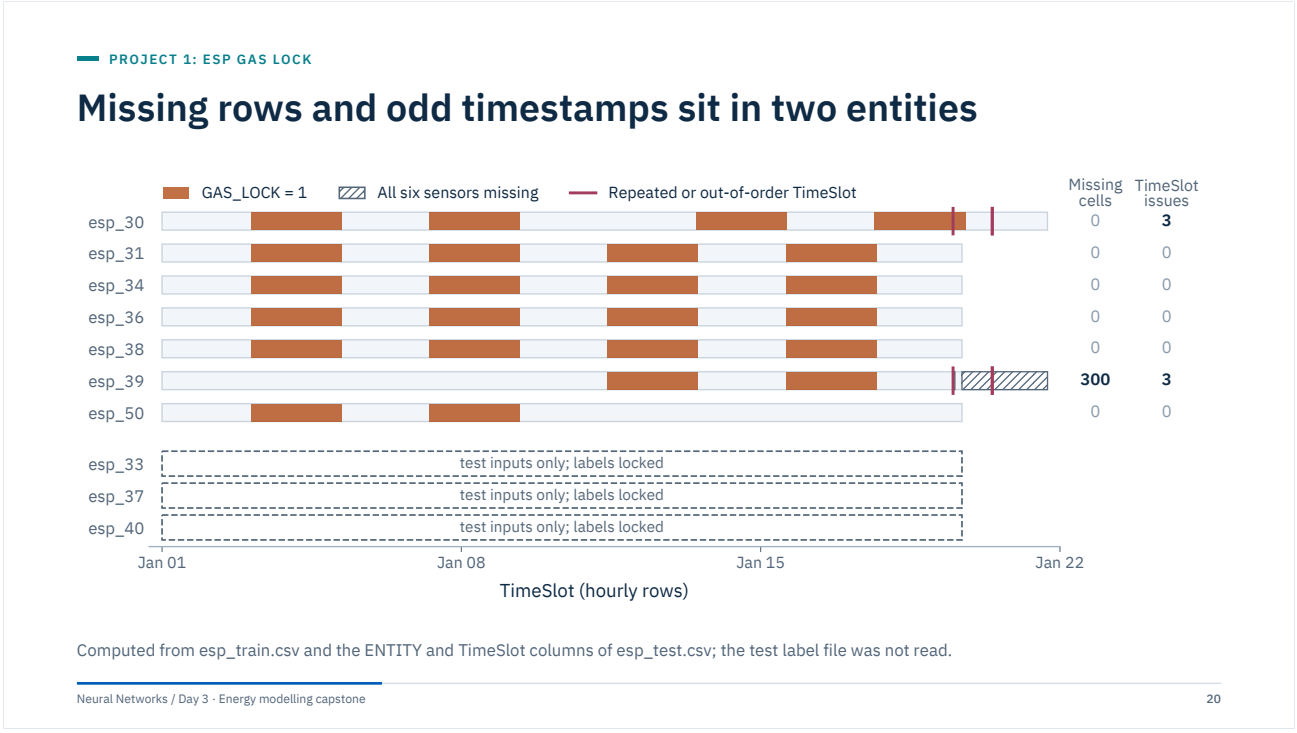


Each spike is an entity's set-point: PIP sits near 450 for esp_30, near 700 for esp_31 and near 1,200 for esp_34 and esp_38, for example. For several sensors the difference between entities is larger than the difference between classes. A model on absolute values can therefore learn "which pump is this" instead of "is this pump gas-locked", which is exactly what the entity holdout punishes.

The gas-lock rows (orange) spread towards lower PDP, Q, THP and Mamps: the ramps from the previous slide. Compare classes within one entity as well as pooled.

An instructor-only audit note: 97.2% of GAS_LOCK = 1 rows (1,190 of 1,224) carry fractional values, against 15.6% of complete GAS_LOCK = 0 rows (307 of 1,969). That is a clue about how the file was produced, not a feature to engineer. If a team discovers it, praise the audit and ask them not to exploit it. [2.5 min]

Missing rows and odd timestamps sit in two entities



Missingness by entity: all 300 missing cells are 50 rows × 6 sensors, the last 50 rows of esp_39 in file order (19-21 January), and all 50 rows are labelled 0. Decide explicitly: drop them and say so, or impute and accept that 50 "normal" rows consist of median values. An indicator column would let a model learn "missing means no gas lock", a shortcut that says nothing about pumps.

Timestamps: esp_30 and esp_39 run two days longer than the others (499 rows) and each contains three repeated or out-of-order TimeSlot values; there are six duplicated entity-time pairs in the file. For row-wise classification this does not break the model, but it must be noted, and any time-based feature would need a fix first.

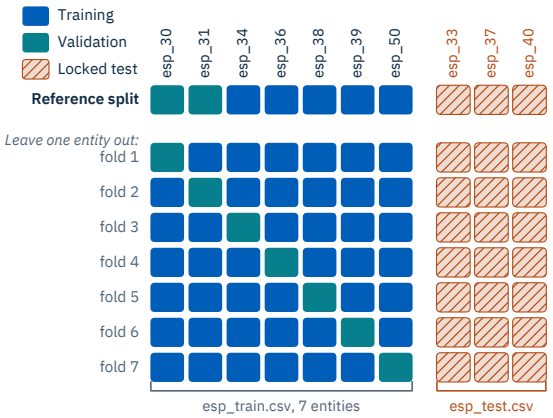
Test entities esp_33, esp_37 and esp_40 cover 1-19 January, the same dates as training. That is why the supplied test assesses unseen entities rather than future time. [2.5 min]

The ESP question is about unseen entities

PROJECT 1: ESP GAS LOCK

The ESP question is about unseen entities

- Can the model classify rows from an entity it has never seen?
- Group validation by ENTITY within the seven training entities
- Keep ENTITY and TimeSlot out of the numeric inputs
- Early warning is a separate task that needs a future target



Computed: GroupShuffleSplit(test_size=2, random_state=42) holds out esp_30 and esp_31; leave-one-entity-out repeats the question seven times.

Neural Networks / Day 3 · Energy modelling capstone 21

Classification using contemporaneous measurements does not establish warning lead time. A forecasting extension must define the target horizon, feature availability at the forecast origin, and chronology; that is Day 5 material.

The reference split holds out two entities, so the validation score depends on which two were drawn. With only seven groups, repeat the grouped evaluation when time permits: leave-one-entity-out gives seven scores and shows the spread between pumps. Report per-entity performance and variation instead of an overconfident aggregate claim.

The three test entities stay locked throughout; they are never part of any fold. [2 min]

Split by entity before fitting any preprocessing

PROJECT 1: ESP GAS LOCK

Split by entity before fitting any preprocessing

```python
from sklearn.model_selection import GroupShuffleSplit

features = ["PIP", "PDP", "Q", "THP", "Mamps", "PIT"]
splitter = GroupShuffleSplit(
    n_splits=1, test_size=2, random_state=42
)
train_idx, val_idx = next(splitter.split(
    train_df[features], train_df["GAS_LOCK"],
    groups=train_df["ENTITY"]
))
val_ents = set(train_df["ENTITY"].iloc[val_idx])
assert not val_ents & set(train_df["ENTITY"].iloc[train_idx])
print(sorted(val_ents), len(train_idx), len(val_idx))
```

OUTPUT

```
['esp_30', 'esp_31'] 2295 948
```

Neural Networks / Day 3 · Energy modelling capstone 22

test_size=2 means two groups here, not two rows and not 2%. The assertion guards the one property that matters: no entity appears on both sides.

The output is what the reference workflow uses: esp_30 and esp_31 (499 + 449 = 948 rows) for validation, the other five entities (2,295 rows) for training. Only after this line are the imputer and scaler fitted, on train_idx rows only.

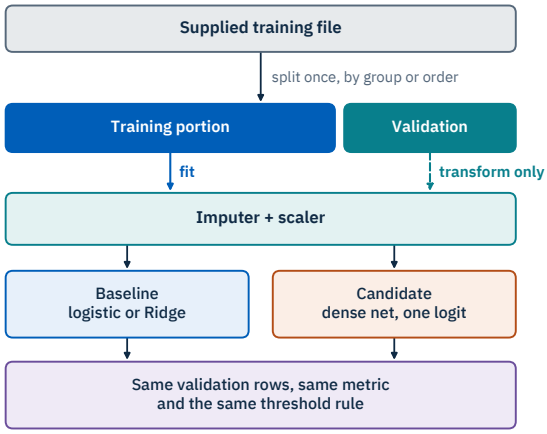
Check label coverage and missingness in both partitions: here the validation entities contain labelled periods (408 positive rows) and no missing values, while esp_39's missing rows fall in training. With only seven groups, repeat grouped evaluation for robustness when time permits. The final supplied test entities remain separate. [2 min]

Baseline and candidate share every other choice

PROJECT 1: ESP GAS LOCK

Baseline and candidate share every other choice

- Baseline: median imputation, scaling, logistic regression
- Candidate: one hidden layer and a raw-logit loss
- Report gas-lock precision, recall and per-entity results
- Add average precision as a threshold-free check



Schematic. Split and fitted preprocessing are shared, so only the model differs.

Neural Networks / Day 3 · Energy modelling capstone 23

The comparison is only fair if the split, the preprocessing and the evaluation rows are identical. The reference workflow guarantees this: both models come from one prepare() call, and the baseline pipeline refits an identical imputer and scaler on the same training rows, so the fitted statistics match. Validation rows are only transformed, never used to fit.

Average precision is a useful threshold-free companion to precision and recall at the chosen threshold. Start without artificial rebalancing, then test class weighting if there is a reason; the experiment-design block shows what weighting does on this file.

Do not promise a target F1 or recall from these files. The candidate should be small enough to debug and reproduce: the reference network 6 → 32 → 1 has 257 parameters.

For orientation, the reference development run (seed 42, validation entities esp_30 and esp_31) gives the baseline an average precision of 0.968 and the network 0.945. That is one split of two entities, not a verdict on either model. [1.5 min]

The ESP final join must be checked

PROJECT 1: ESP GAS LOCK

The ESP final join must be checked

```
scored = test_df[["uuid"]].copy()
scored["probability"] = probabilities
scored = scored.merge(
    actual_df, on="uuid", how="left", validate="one_to_one"
)
assert len(scored) == len(test_df)
assert scored["GAS_LOCK"].notna().all()
```

Run this once, at milestone 4, after the configuration is locked.

⚠ A join that leaves missing labels means the files do not correspond: stop, do not drop rows.

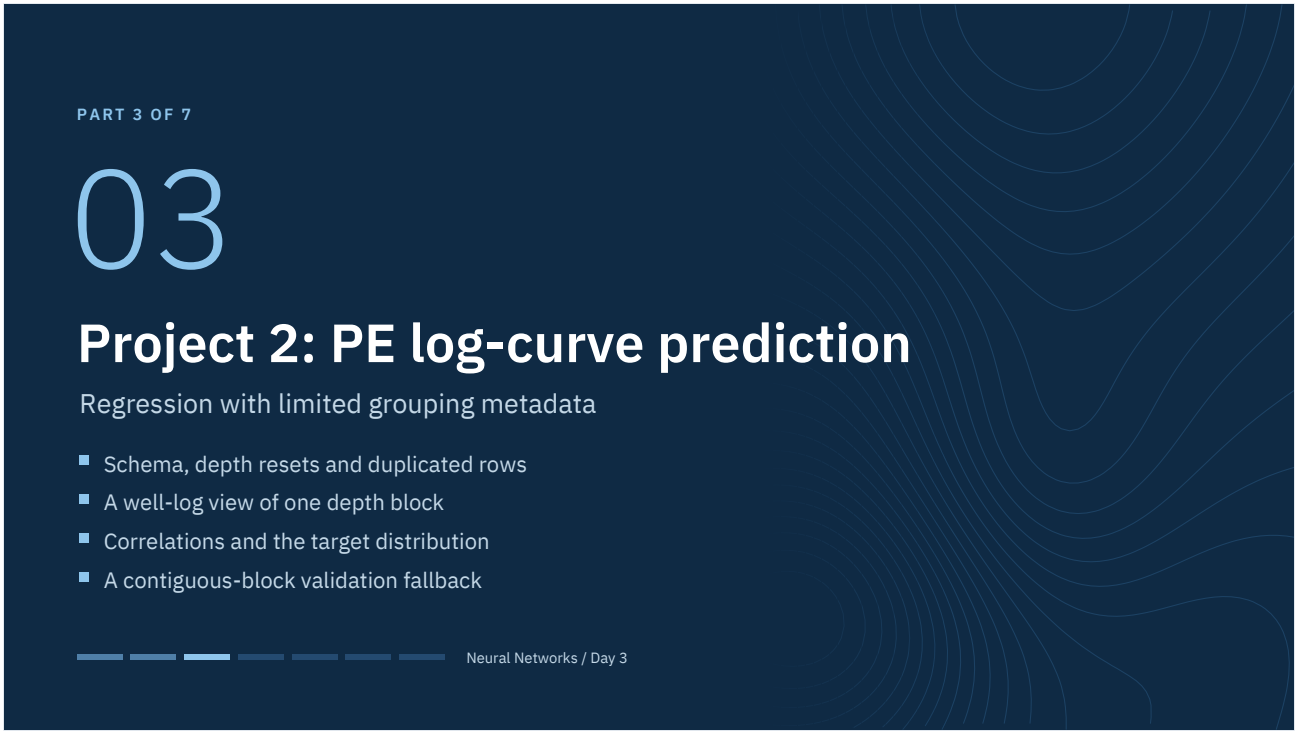
Neural Networks / Day 3 · Energy modelling capstone 24

probabilities must preserve test_df row order before the join; compute them from test_df directly and never reorder in between. Verify uuid uniqueness in both tables (validate="one_to_one" raises if either side has duplicates). Do not fit or tune using actual_df.

Missing matches or duplicate keys are errors, not values to impute: a left join that produces NaN labels means the files do not correspond, and scoring after dropping those rows would silently change the evaluation set.

The reference workflow performs the same checks in final_data(): it raises "Duplicate UUIDs" or "Label alignment failed" rather than continuing. [1.5 min]

Project 2: PE log-curve prediction



Use the observed schema throughout the lab: the inputs are Depth, GR, ILD_log10, DeltaPHI, PHIND and RELPOS, and the target is PE. No well-ID column is present, so this dataset alone cannot support a documented leave-one-well-out claim. Everything in this part is about making that limit explicit while still running a defensible experiment. Geoscientists in the room will know PE as a lithology-sensitive log; keep physical interpretation out of the evidence unless the data owner confirms units and acquisition. [0.5 min]

Six inputs and PE, but no well identifier

PROJECT 2: PE LOG-CURVE PREDICTION

Six inputs and PE, but no well identifier

Role	Columns	Known limitation
Predictors	Depth, GR, ILD_log10	Units and acquisition grouping need documentation
Predictors	DeltaPHI, PHIND, RELPOS	Use provided definitions when available
Training target	PE	Numeric regression target
Supplied evaluation	310 input rows and 310 PE labels	Label alignment has no explicit key

◆ Depth is supplied as a column, not proven to be measured depth in a single well.

Neural Networks / Day 3 · Energy modelling capstone 26

The training file contains 2,922 rows and seven columns including PE, with no missing values. The PE range is 0.2 to 8.094 in the training file (median 3.51). A range is a file audit, not a physical validity criterion. Preserve original row order in evaluation.

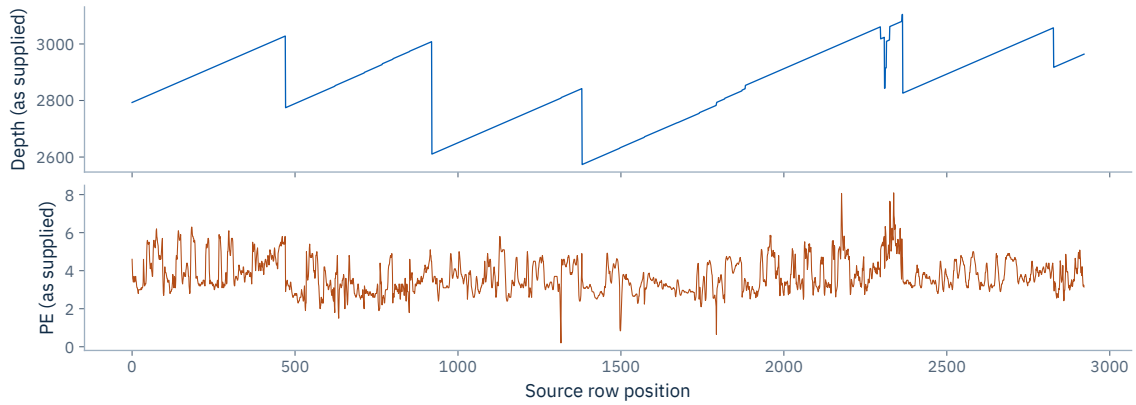
The names suggest gamma ray, log10 of deep resistivity, a porosity difference, an average porosity and a relative position within a unit, but the file does not define them. Treat Depth as supplied: it is not guaranteed to be measured depth in a single well, as the next slide shows.

Ask: which column is most dangerous as a predictor? Depth, because it can encode position within a particular block of rows rather than rock properties. It is allowed here because the reference contract includes it, but teams should test its effect in their experiment if they suspect it. [2 min]

Depth resets split the file into eight blocks

PROJECT 2: PE LOG-CURVE PREDICTION

Depth resets split the file into eight blocks



Supplied log training data, plotted in source-row order. The file lacks well identifiers, so changes in depth cannot alone establish well boundaries.

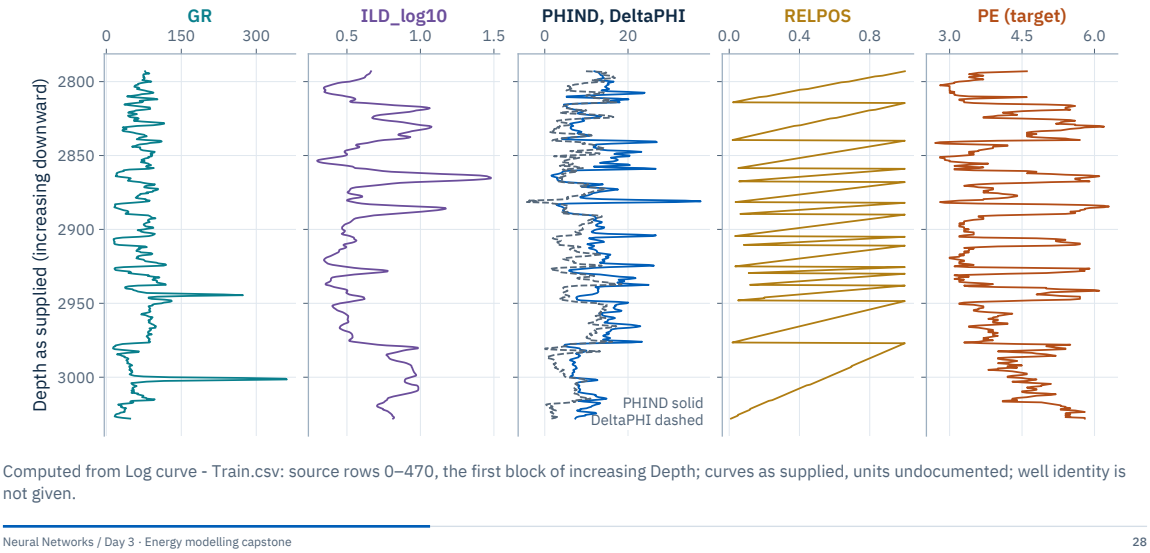
Neural Networks / Day 3 · Energy modelling capstone 27

Ask learners to inspect depth resets, repeated depths, and changes in distributions. Depth decreases seven times in source-row order, so the file consists of eight blocks of increasing depth. Most steps are 0.5 units; three steps are zero (repeated depths), and a few are larger gaps.

Two audit findings to record. Rows 2,297-2,308 repeat rows 450-461 exactly (a duplicated 12-row block), and the short blocks around rows 2,297-2,364 jump in depth, which suggests stitched fragments.

Do not simply sort all rows by depth and pretend they form one well. Source-row blocking is a transparent teaching fallback but not proof of spatial or cross-well generalization. [2 min]

Read one depth block as a well log first



A track plot is the fastest audit a geoscientist can do: depth down the page, one curve per track. Look for spikes (GR reaches 361 in the file), flat or clipped sections, and whether PE moves with the porosity and resistivity curves.

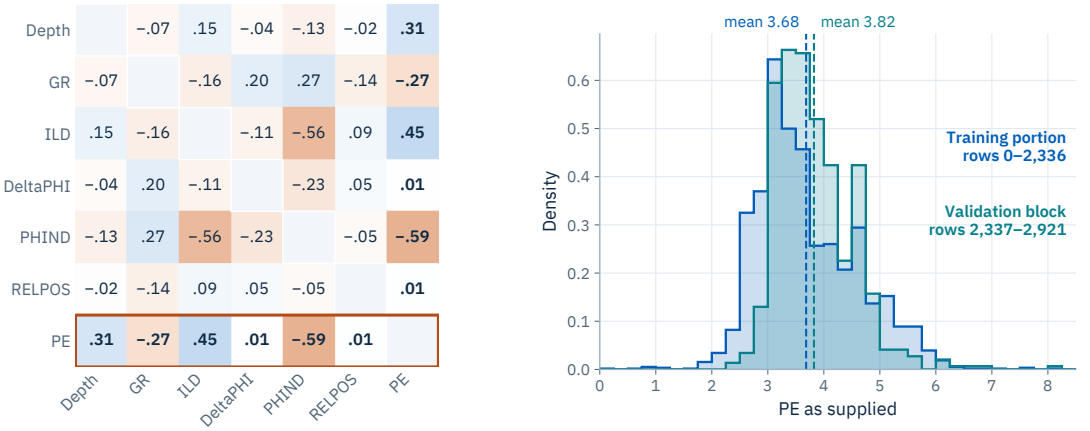
In this block, PE rises where PHIND falls (r = −0.70) and ILD_log10 rises (r = +0.71). It also falls with DeltaPHI (r = −0.52), although over the whole file that correlation is near zero: relationships differ between blocks, another reason not to pool rows blindly. RELPOS is a sawtooth from 1 at the top of each unit to about 0 at its base; its definition is not supplied, so describe it rather than interpret it.

Ask: what would convince you that two blocks come from the same well? Matching depth ranges alone would not; you would need an identifier or overlapping, identical curve segments. [2 min]

PE follows PHIND and ILD; validation PE runs higher

PROJECT 2: PE LOG-CURVE PREDICTION

PE follows PHIND and ILD; validation PE runs higher



Computed from Log curve - Train.csv. Left: Pearson correlations over all 2,922 rows (ILD = ILD_log10; descriptive, rows are not independent). Right: PE in the training portion and the validation block of the reference split.

Neural Networks / Day 3 · Energy modelling capstone 29

Left: PE correlates most with PHIND (−0.59) and ILD_log10 (+0.45), then Depth (+0.31) and GR (−0.27); DeltaPHI and RELPOS are near zero linearly. A near-zero linear correlation does not make a column useless to a network, and a strong one does not prove a causal link. Do not select features from a matrix computed on all rows, which includes the validation block; use it to understand the file.

Right: the validation block (rows 2,337-2,921) has a higher mean PE (3.82) than the training portion (3.68) and a different shape. A model that predicts the training mean everywhere therefore scores a negative R^2 on this block (−0.04). That is not a bug: R^2 compares with the validation mean, which the model never saw.

Target scaling in the reference workflow is fitted on the training portion only, and predictions are converted back to PE units before scoring. [3 min]

A contiguous source-row block is the honest fallback

PROJECT 2: PE LOG-CURVE PREDICTION

A contiguous source-row block is the honest fallback

- Hold out a contiguous block in source-row order
- Fit input and target scalers on the remaining rows only
- Compare with the training mean and a Ridge baseline
- Asset-level generalization stays unverified without well IDs



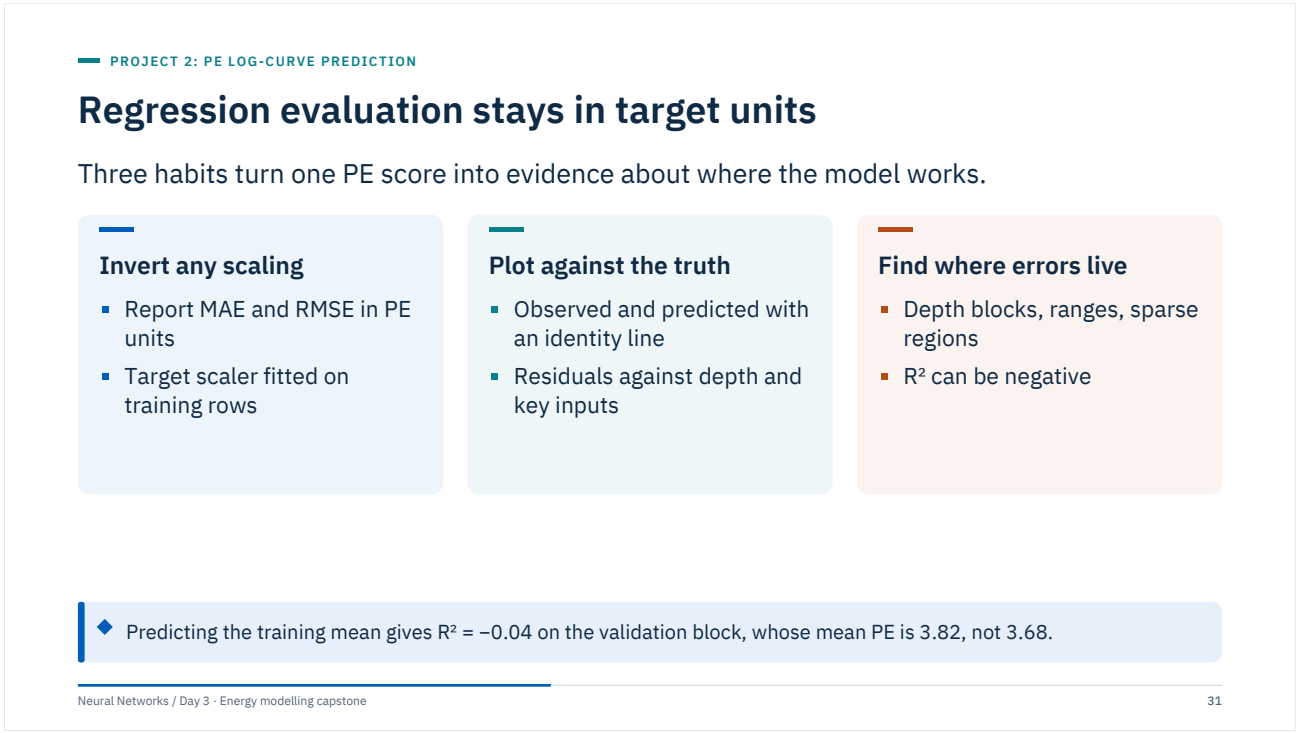
Computed from Log curve - Train.csv: the reference split keeps rows 0–2,336 for training and holds out rows 2,337–2,921.

The supplied row ordering is not documented as time or well order. A contiguous block is a conservative reproducible exercise choice, not a guaranteed leakage-free geologic split: a random row split would put neighbouring depth samples (0.5 units apart, nearly identical logs) on both sides and flatter every model.

An instructor observation from the test inputs: the test file's Depth starts at 2,964.5, directly below the last training row (2,964.0), and RELPOS continues its decline. That is consistent with the test being the deeper continuation of the final training block, which makes a late source-row block a sensible rehearsal, but nothing in the files confirms it.

If well IDs become available, replace this with grouped evaluation. Discuss this limitation prominently in the final presentation. [2.5 min]

Regression evaluation stays in target units



Invert any target scaling before reporting MAE or RMSE; the reference workflow's learning curve is in standardized units, but its metrics are in PE units. Plot observed and predicted PE together with an identity line, and residuals against depth and important predictors.

R^2 can be negative: on the reference validation block the training-mean prediction scores $R^2 = -0.04$ and the Ridge baseline −0.08, with MAE 0.517 and 0.546. A single aggregate score can hide poor behaviour in a particular interval, which the error-analysis part demonstrates on this block.

Do not label residuals in microseconds per foot: this task predicts PE, not sonic slowness. Avoid attaching unverified units. [1.5 min]

Positional labels require row-order discipline

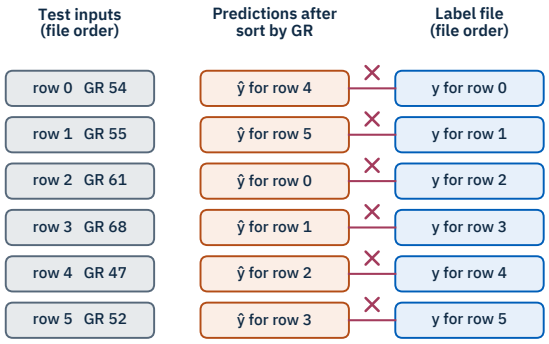
PRACTICE

4 min · Pairs

Positional labels require row-order discipline

You sort the PE test inputs by GR to inspect outliers, predict, and score against the unchanged label file.

1. What happens to the score?
2. Why does the row-count check still pass?
3. How should the evaluation preserve alignment?



Row counts match (310 = 310), so the length check passes, yet every prediction is scored against another row's label.

Schematic with illustrative GR values.

Neural Networks / Day 3 · Energy modelling capstone 32

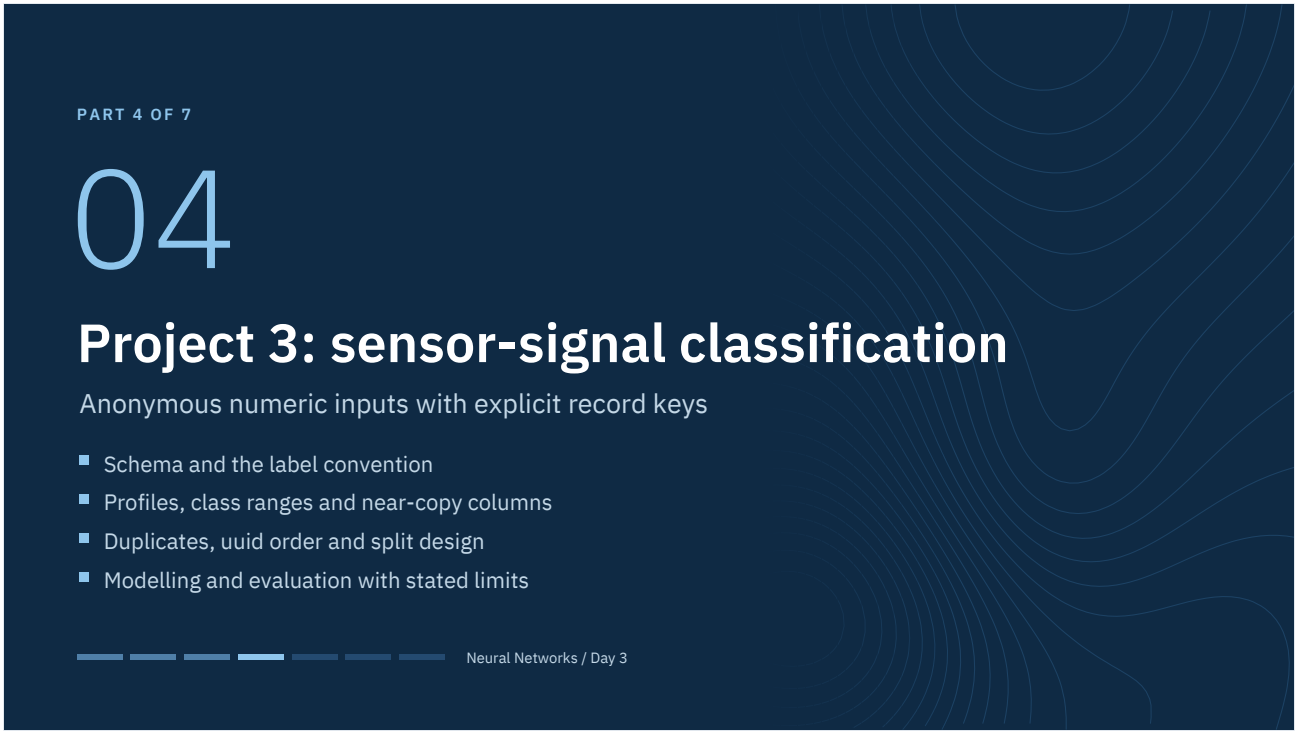
Answers.

1. Predictions and targets now refer to different rows, so the score measures nothing. It usually collapses towards the score of a shuffled target. An illustration on the training file's validation block (not the test): scoring Ridge predictions in GR order against labels in file order raises MAE from 0.546 to 0.649 and drops R^2 from −0.08 to −0.43; only 1 of 585 pairs stays aligned. 2. Sorting changes the order, not the count: 310 predictions still meet 310 labels. A matching row count is necessary but does not prove semantic alignment. 3. Keep the original row index, restore it before scoring (sort_index), or never reorder the evaluation frame; assert that the index equals range(310) before scoring. The reference final_data() uses the unchanged file order and checks the count.

Twist for fast pairs: in the supplied test file Depth already increases monotonically, so a Depth sort would happen to be harmless. That is luck, not design; you can only know it by checking.

Common error: calling the misaligned score "a bit worse". It is not a noisier estimate of the same quantity; it measures nothing. [4 min + 2 min debrief]

Project 3: sensor-signal classification



Keep the problem statement faithful to the files. The sensor columns are col1 through col10. Their physical meaning, units, and temporal ordering are not established by their names. This project is the purest test of the day's theme: with no domain story to lean on, the evidence has to come from the data audit and the split.

This briefing runs after the break (10:30-10:45). [0.5 min]

Ten anonymous columns, a key and ±1 labels

PROJECT 3: SENSOR-SIGNAL CLASSIFICATION

Ten anonymous columns, a key and ±1 labels

```
target = train_df["target"]
assert set(target) == {-1, 1}
y = (target == 1).astype(int)
X = train_df[[f"col{i}" for i in range(1, 11)]]
assert train_df["uuid"].is_unique
```

- Ten predictors: col1 through col10
- uuid identifies records and joins test labels
- Labels are −1 and 1, not 0 and 1
- Map them explicitly before a binary-logit loss

⚠ BCEWithLogitsLoss accepts a −1 target without error and silently optimizes the wrong objective.

Neural Networks / Day 3 · Energy modelling capstone 34

Training contains 2,061 labels equal to 1 and 1,519 equal to −1 (57.6% ones). Map −1 to 0 and 1 to 1 explicitly for a binary-logit loss; BCEWithLogitsLoss with a target of −1 does not raise an error, it silently optimizes the wrong objective. Map predictions back to the original −1/1 convention when exporting labelled results.

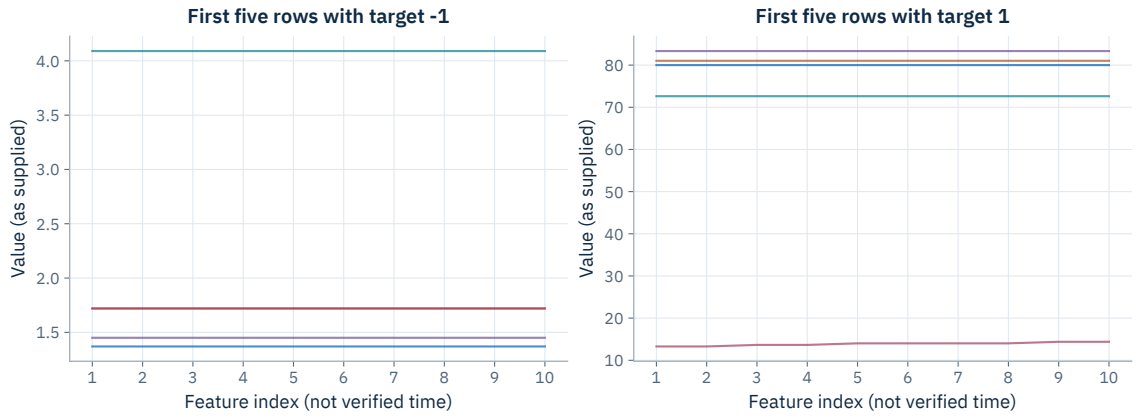
uuid is unique in both the training and test files and is the join key for Sensor_target.csv; it is not a predictor.

Verify unique keys and decide whether duplicated feature vectors indicate repeated or related measurements: the next slides show there are many. Do not invent normal/fault semantics without a data dictionary. [1.5 min]

Inspect the profiles before choosing an architecture

PROJECT 3: SENSOR-SIGNAL CLASSIFICATION

Inspect the profiles before choosing an architecture



Selected rows from the supplied sensor training file. The horizontal axis is feature index, not verified elapsed time.

Neural Networks / Day 3 · Energy modelling capstone 35

The figure helps inspect similar or flat profiles. Most of these rows are perfectly flat: ten identical values. Two of the five target –1 rows are identical (1.72 in every column), so the left panel shows only four lines: a first glimpse of the duplicates. One row rises slowly across the index. The two panels also have very different vertical scales, a first hint that the classes sit in different value ranges.

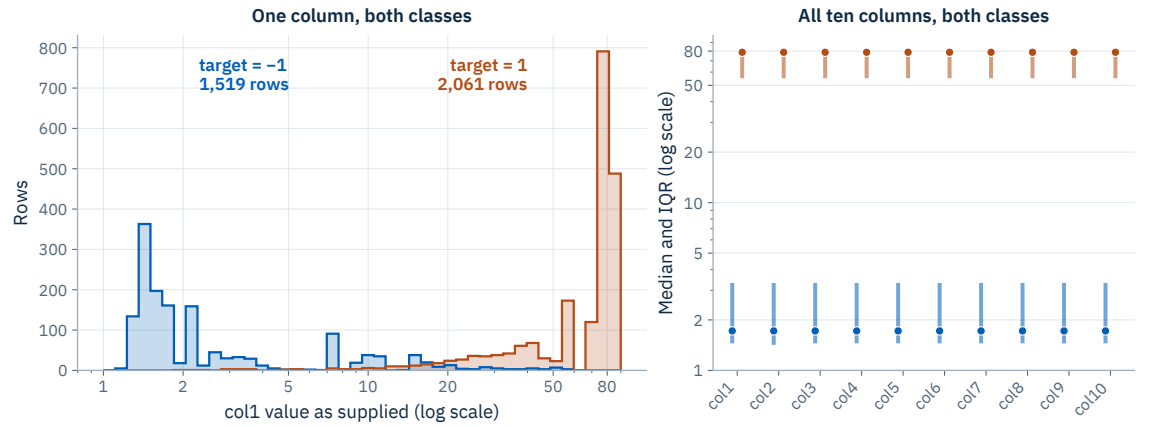
Consecutive column names do not prove a valid sequence-model interpretation. A 1D convolution or recurrent network (Days 4 and 5) assumes an ordered axis; nothing here documents one. Use a dense baseline first.

If duplicate profiles appear across partitions, investigate whether evaluation independence is compromised; three slides from now we quantify exactly that. [1.5 min]

The two classes occupy different value ranges

PROJECT 3: SENSOR-SIGNAL CLASSIFICATION

The two classes occupy different value ranges



Computed from Sensor_train.csv (3,580 rows). Column index is a name, not a verified time axis.

Left: target −1 rows concentrate at small values (median 1.72) and target 1 rows at large values (median 78.74). The ranges do overlap (target −1 reaches 55.4 and target 1 goes down to 1.9 in col1), but 95% of target −1 rows lie below 16.5 and 95% of target 1 rows above 19.2. Right: the medians and interquartile ranges are practically identical for all ten columns, so each column carries almost the same class information.

Consequences for the playbook. A linear baseline is strong on this file, and a network has little room to improve on it. That is a property of these data, not a general statement about sensors.

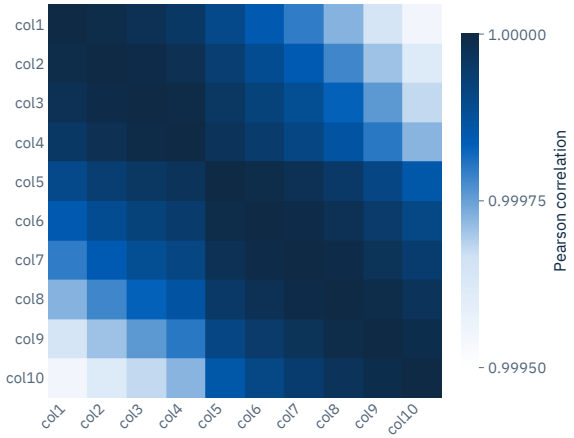
Ask: if the classes were separated this cleanly in deployment, what would still worry you? Expected answers: whether the value ranges are stable over time or across devices, and whether the validation rows are independent of the training rows. The next two slides answer the second question. [2.5 min]

The ten columns are near-copies of one another

PROJECT 3: SENSOR-SIGNAL CLASSIFICATION

The ten columns are near-copies of one another

- Every pairwise correlation is at least 0.9995
- 65% of rows hold ten identical values, in both classes
- Treat the columns as one signal plus small deviations
- No data dictionary supports reading the index as time



Computed from Sensor_train.csv: Pearson correlations between col1–col10 over 3,580 rows.

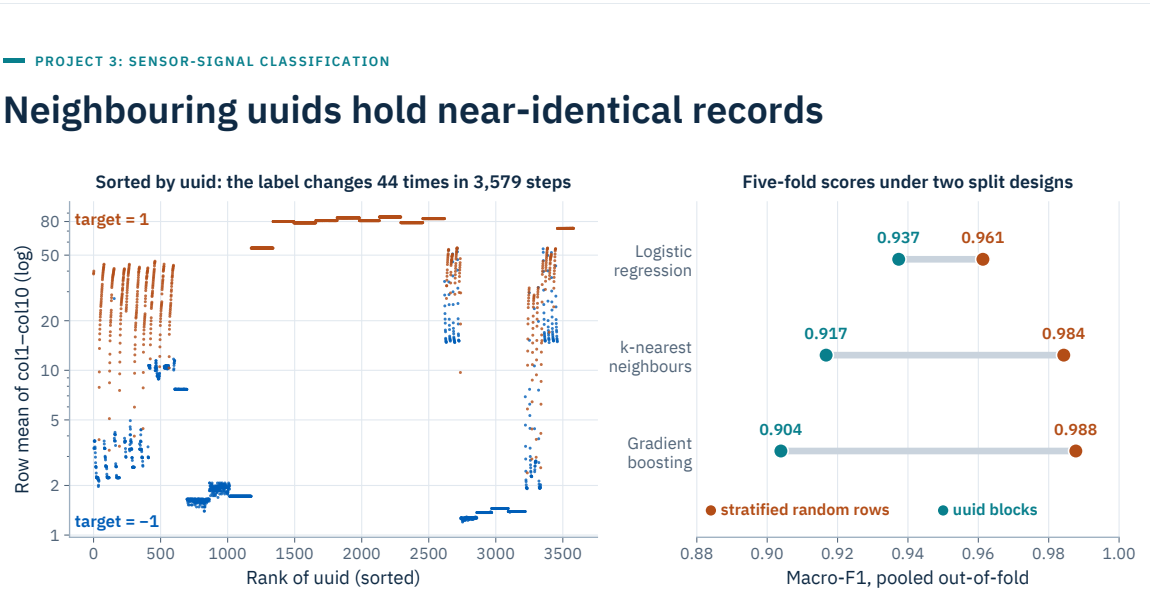
Neural Networks / Day 3 · Energy modelling capstone 37

The minimum off-diagonal correlation is 0.99955; the heat map needs a colour scale from 0.9995 to 1 to show any structure at all. 2,328 of 3,580 rows (65.0%) have ten identical values; the share is 64.9% for target −1 and 65.1% for target 1, so flatness does not discriminate the classes.

Where a row varies, the values change gradually across the column index. That is consistent with an ordered measurement, but nothing documents it, so do not build a temporal story on it.

Practical consequences: a dense network sees ten nearly collinear inputs, which is harmless for prediction but makes per-column importance meaningless (ML Module 2 covers correlated features in importance measures). A single summary feature such as the row mean is a legitimate controlled experiment for this project. [2.5 min]

Neighbouring uuids hold near-identical records

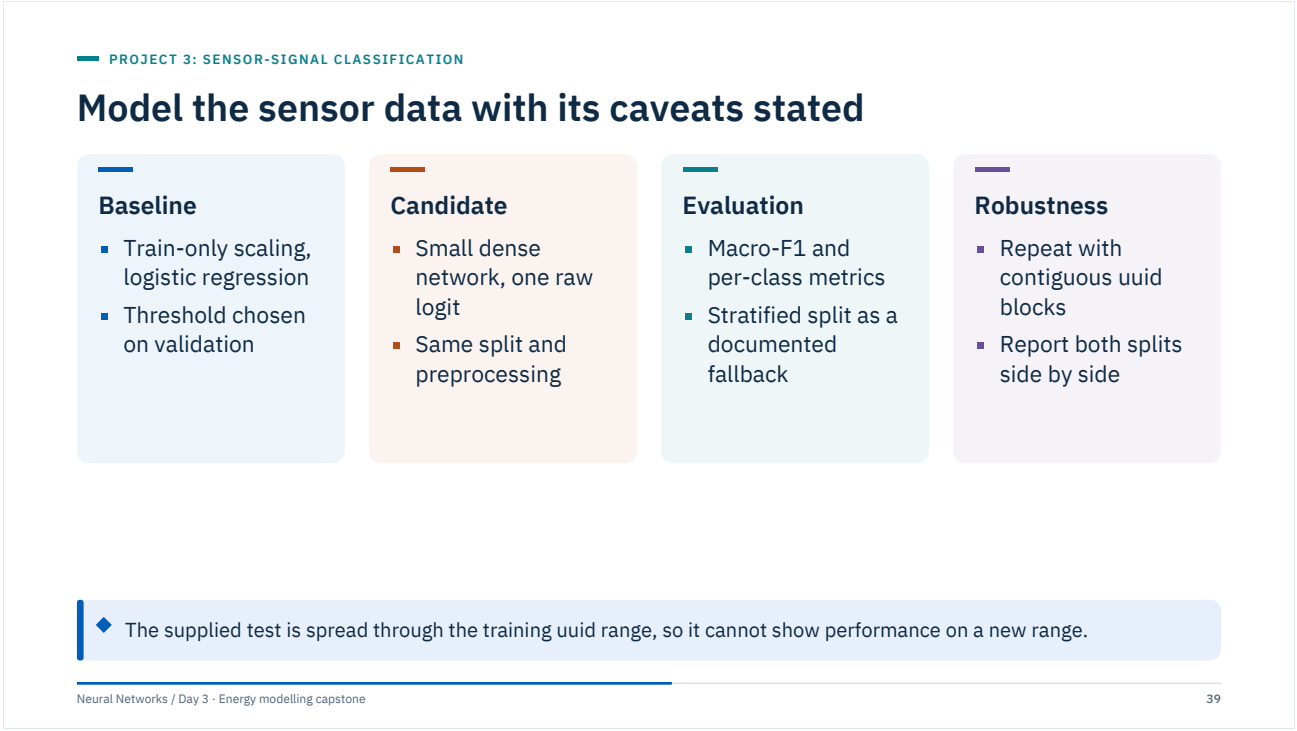


Left: sorted by uuid, records form long runs of near-identical values and the label changes only 44 times in 3,579 steps. 2,484 rows repeat an earlier row's feature vector exactly (1,096 distinct vectors), and in the reference stratified split 530 of the 716 validation rows (74%) have an exact duplicate in the training portion.

Right: holding out contiguous uuid blocks lowers macro-F1 for every model, and most for the flexible ones: logistic regression 0.961 → 0.937, k-nearest neighbours 0.984 → 0.917, gradient boosting 0.988 → 0.904. The ranking flips, as it did for ESP. Grouping only exact duplicates changes nothing (logistic regression stays at 0.961), because near-duplicates remain.

The supplied test is spread through the same uuid range (its uuids interleave with the training uuids, about 20% of every fifth of the range, and 646 of 895 test rows repeat a training row), so it resembles the random split: it cannot show performance on records from a new uuid range. The meaning of uuid order is undocumented; use the block split as a robustness check, not as proof. [3.5 min]

Model the sensor data with its caveats stated



Stratification preserves class proportions but does not establish independence. A robustness extension keeps exact duplicate profiles in one group; on this file the contiguous uuid-block split is the more informative check, because near-duplicates survive exact-duplicate grouping.

For orientation, the reference development run gives macro-F1 0.966 for the baseline and 0.963 for the network on the stratified validation rows, at thresholds chosen on validation. Differences in the third decimal are not evidence of anything without seed repeats.

Map predictions back to the original label convention when exporting labelled results. Join final targets using uuid with a one-to-one check. [2 min]

Sprint 1: write the data contract and audit

LABORATORY

35 min

Sprint 1: write the data contract and audit

- ☐ Form a team of two or three; open your project's student notebook
- ☐ Notebook sections 1–2: inspect the file and write the prediction contract
- ☐ Audit missing values, duplicates, keys, groups and order with the checklist
- ☐ Sections 3–4: check the split and the training-only preprocessing

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- One-page data contract
- Audit table with a decision per finding
- Split policy and what it cannot show

Neural Networks / Day 3 · Energy modelling capstone 40

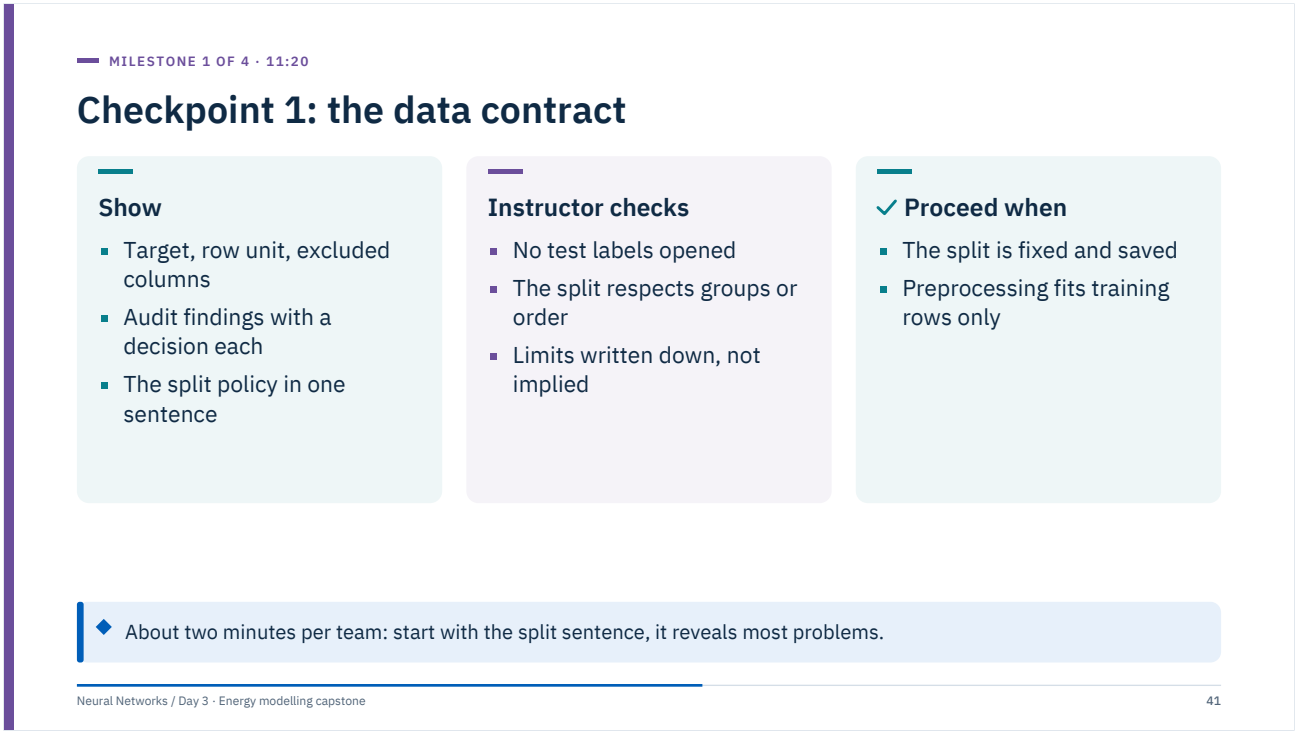
Notebooks: exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb, exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb, exercises/03_sensor_signals/Sensor_Signals_Student.ipynb. All three import prepare() and run() from curriculum/capstone.py.

Timing: 0-5 min teams and project; 5-15 schema and contract (section 2 is a written cell); 15-25 audit using the checklist slide; 25-35 sections 3-4 (split policy and the invariant check: imputer medians equal medians of training rows only; for ESP, assert disjoint entities).

Worked answer for the section 4 invariant: np.allclose(data["prep"][0].statistics_, data["X"].iloc[data["train_idx"]].median()) returns True. Computing the medians on all rows would include validation rows and fail the check whenever the medians differ.

Common errors: opening a test label file "just to look"; listing ENTITY or uuid as predictors; sorting the PE file by Depth; forgetting the −1/1 mapping; writing units guessed from column names. Circulate and ask each team for its limit sentence before 11:20. [35 min]

Checkpoint 1: the data contract



About two minutes per team at its own screen. Ask for the split sentence first; it reveals most problems. Good examples: "Validation holds out esp_30 and esp_31; this tests unseen entities, not future time." "Validation is source rows 2,337-2,921; without well IDs this is not a cross-well test." "Validation is a stratified 20% of rows; 74% of them duplicate a training row, so we will also report a uuid-block split."

Typical findings to praise: the 50 all-missing esp_39 rows, the duplicated 12-row PE block, the sensor duplicates. Typical problems: preprocessing fitted before splitting, a random row split for ESP, labels guessed as physical faults.

A team that is not ready keeps the contract as its first task in Sprint 2; revisit it at the start of that sprint (11:50). [10 min for the room]

Baselines and experiments



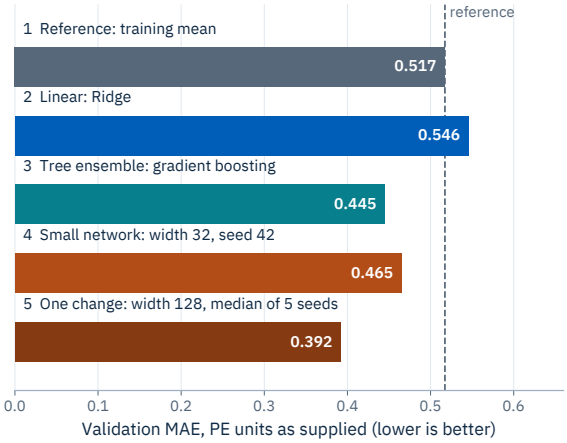
This part is taught in two short blocks: the tabular-NN playbook before Sprint 2 (11:30-11:50) and experiment design after lunch (13:25-13:40). The material is a practical recap of Days 1 and 2 applied to small tabular data, plus an honest look at why tree ensembles are hard to beat on such data. Ask the room before starting: on which of the three projects do you expect a network to help most, and why? Revisit the answers at checkpoint 3. [1 min]

The playbook starts with the cheapest honest model

BASELINES AND EXPERIMENTS

The playbook starts with the cheapest honest model

1. **Reference**: training mean or class prevalence
2. **Linear**: logistic or Ridge regression
3. **Tree ensemble**: gradient boosting
4. **Small network**: one hidden layer
5. **One change**: a controlled experiment



Computed on the PE reference split (source rows 0–2,336 train, 2,337–2,921 validation).

Neural Networks / Day 3 · Energy modelling capstone 43

Each rung must beat the rung above it on the same validation rows to earn its complexity. The PE numbers show why every rung matters. The Ridge baseline (MAE 0.546) is worse than simply predicting the training mean (0.517); without rung 1 nobody would notice. Gradient boosting (0.445) beats the default network run (0.465). A controlled change to width 128 brings the network to a median of 0.392 over five seeds.

These are validation scores on one contiguous block, so they are selected estimates and say nothing about other wells.

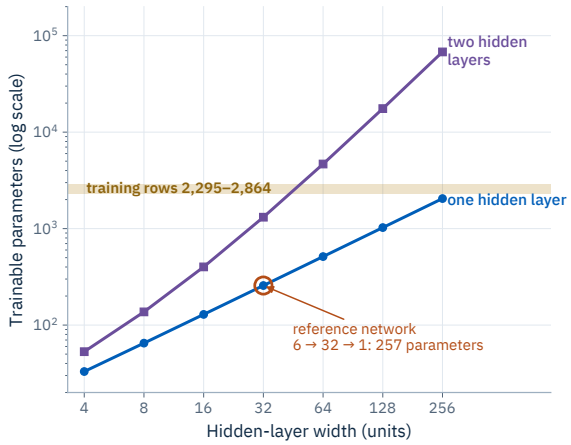
The reference workflow in curriculum/capstone.py implements rungs 2 and 4. Rung 1 is one line (the training mean or prevalence), and rung 3 is a few lines with HistGradientBoostingRegressor or HistGradientBoostingClassifier, which also accept missing values directly. [3 min]

A few thousand rows call for a small network

BASELINES AND EXPERIMENTS

A few thousand rows call for a small network

- Start with one hidden layer of 16–64 units
- 6 → 32 → 1 has 257 parameters; two layers of 64 have 4,673
- Parameter count is not a hard limit; validation decides
- Width is cheap to test; depth adds tuning burden



Exact parameter counts for dense networks with 6 inputs and one output; the band spans the training rows of the three reference splits.

Neural Networks / Day 3 · Energy modelling capstone 44

Worked count for the reference network: weights 6 × 32 = 192, hidden biases 32, output weights 32 × 1 = 32, output bias 1, total 257. With 10 sensor inputs it is 10 × 32 + 32 + 32 + 1 = 385. Two hidden layers of 64: 6 × 64 + 64 + 64 × 64 + 64 + 64 + 1 = 4,673.

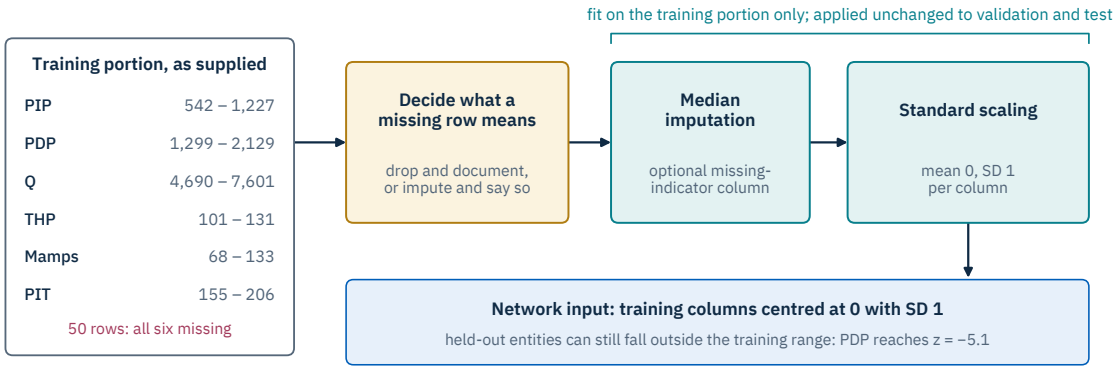
The reference splits train on 2,295 (ESP), 2,337 (PE) and 2,864 (sensor) rows. Networks with more parameters than rows can still generalize, especially with early stopping and weight decay (Day 2), but they are slower to debug and more sensitive to seeds.

Ask: what should decide the width? The validation curve over several seeds, compared with the baselines. On the PE block, the seed spread at width 32 turned out to be larger than the gain from width 32 to 128, which is the subject of the controlled-experiment slides. [3 min]

Scale inputs and decide what a missing row means

BASELINES AND EXPERIMENTS

Scale inputs and decide what a missing row means



Schematic; value ranges computed from the training portion of the reference ESP split in esp_train.csv.

Neural Networks / Day 3 · Energy modelling capstone 45

Scaling matters for networks because gradient steps are shared across all weights: Q spans thousands while THP spans tens, so without scaling the loss surface is badly conditioned and the learning rate that suits one input is wrong for another. StandardScaler fitted on the training portion fixes the scale; it is applied unchanged to validation and test rows.

Scaling does not guarantee a bounded input. Held-out entities can fall far outside the training range: PDP in the ESP validation entities reaches z = −5.1. The network then extrapolates, which is one reason the ESP holdout is hard.

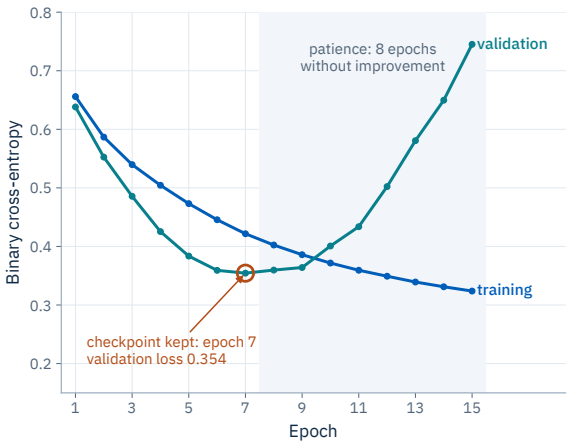
Missing values: SimpleImputer(add_indicator=True) appends indicator columns. In ESP all six sensors go missing together in 50 rows that are all labelled 0, so an indicator would teach "missing means normal", a shortcut about this file rather than about pumps. Either drop the rows and document it, or impute without an indicator and say so. The reference workflow imputes medians without indicators. [3 min]

Early stopping keeps the best validation checkpoint

BASELINES AND EXPERIMENTS

Early stopping keeps the best validation checkpoint

- Validation loss is lowest at epoch 7, then climbs
- The loop copies that state; patience is 8 epochs
- Restore the copy before evaluating or saving
- Training loss keeps falling: the gap is the warning



Computed: curriculum/capstone.py on the ESP project (width 32, AdamW, lr 0.001, seed 42); validation = esp_30 and esp_31.

Neural Networks / Day 3 · Energy modelling capstone 46

This is a recap of Day 2 on today's data. The reference loop evaluates validation loss after every epoch, keeps copy.deepcopy(model.state_dict()) whenever it improves, and stops after eight epochs without improvement. Here the best validation cross-entropy is 0.354 at epoch 7; by epoch 15 it has risen to 0.745 while training loss is still falling (0.324).

The widening gap is the ESP entity story again: the network is fitting the five training pumps' set-points, which do not transfer to esp_30 and esp_31.

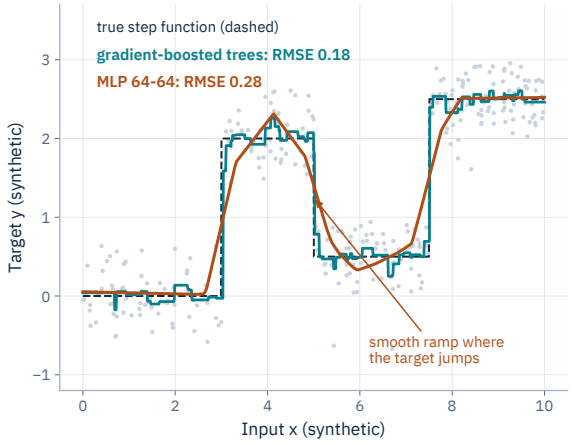
Two classic bugs: a shallow copy (state_dict() without deepcopy) keeps a reference that later updates overwrite, and forgetting load_state_dict(best_state) evaluates the last epoch instead of the selected one. The reference workflow also checks that predictions are identical after saving and reloading. [3 min]

Tree ensembles are often strong on tabular data

BASELINES AND EXPERIMENTS

Tree ensembles are often strong on tabular data

- A 45-dataset benchmark found trees ahead on medium-sized tables
- Networks favour smooth functions; trees fit jumps readily
- Uninformative columns hurt MLPs more than trees
- Columns carry meaning; trees split along them



Synthetic example, seed 0: 300 noisy samples of a step function; RMSE against the true function.

Grinsztajn, Oyallon and Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?", NeurIPS 2022 Datasets and Benchmarks Track (arXiv 2207.08815). Their abstract reports that on a standard set of 45 tabular datasets, tree-based models such as XGBoost and random forests remain state-of-the-art on medium-sized data (around 10,000 samples), even without accounting for their speed. They attribute the gap to inductive biases and state three challenges for tabular networks: be robust to uninformative features, preserve the orientation of the data, and learn irregular functions easily.

The figure illustrates only the third point, on synthetic data: gradient-boosted trees recover the step function (RMSE 0.18 against the truth) while a 64-64 MLP rounds every jump into a ramp (RMSE 0.28). It is a demonstration of a mechanism, not a reproduction of the paper's benchmark.

For our files, "medium-sized" means even smaller: 2,300-2,900 training rows. [3 min]

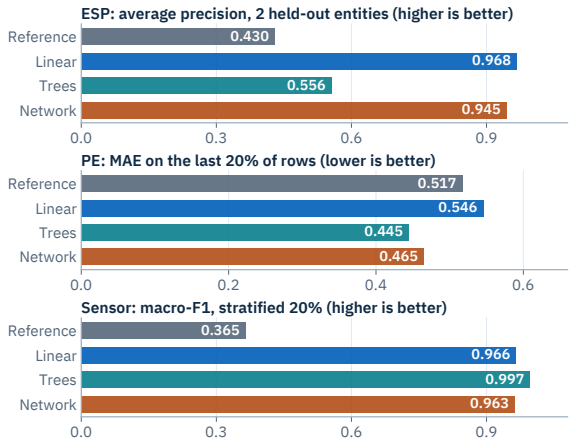
Reference: https://arxiv.org/abs/2207.08815

Treat a tree ensemble as a second baseline

BASELINES AND EXPERIMENTS

Treat a tree ensemble as a second baseline

- Fit gradient-boosted trees on the same rows; they accept missing values
- Trees win on PE and on sensor rows, but fail on held-out ESP pumps
- Rank models per project and per split, not by reputation
- The network does not need to win; a clear negative result counts



Computed on the reference splits (training files only). Network: width 32, seed 42. Sensor thresholds chosen on validation.

Neural Networks / Day 3 · Energy modelling capstone 48

Read the figure per project; within each panel every bar is scored on the same validation rows. PE: trees beat the default network (MAE 0.445 against 0.465), and both beat the training mean (0.517). Sensor: trees reach macro-F1 0.997 on the stratified rows, but the uuid-block check earlier dropped them to 0.904 (threshold 0.5), below logistic regression. ESP: on the two held-out pumps (prevalence 0.430) trees reach an average precision of only 0.556 against 0.968 for logistic regression; splits on absolute set-points do not transfer to new pumps.

So "trees are strong on tabular data" is a prior, not a law: the benchmark asks a different question from our entity and block holdouts.

Code for the tree rung: from sklearn.ensemble import HistGradientBoostingClassifier tree = HistGradientBoostingClassifier(random_state=0) tree.fit(X_train, y_train); p = tree.predict_proba(X_val)[:, 1] Use the rows data["train_idx"] and data["val_idx"] from the reference workflow. [3 min]

Sprint 2: fit the baselines and a first network

LABORATORY

40 min

Sprint 2: fit the baselines and a first network

- [ ] Section 5: run the reference baseline and network on your split
- [ ] Add a no-skill reference and a tree ensemble on the same rows
- [ ] Section 6: read the learning curve and note the selected epoch
- [ ] Save the run folder; record the command, seed and versions

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table: reference, linear, tree, network
- Learning curve with the selected epoch
- The exact command that reproduced it

Neural Networks / Day 3 · Energy modelling capstone 49

Timing: 0-10 min run section 5 (seconds on a laptop CPU) and inspect results.json; 10-22 add the reference and the tree ensemble using data["train_idx"] and data["val_idx"]; 22-30 learning curve (section 6); 30-40 fresh-process rerun and the evidence log.

What teams should see in the reference development run (seed 42, width 32, 40 epochs, course environment): ESP baseline AP 0.968 and network 0.945 (best epoch 7, thresholds 0.775 and 0.45); PE Ridge MAE 0.546 (R^2 −0.08) and network 0.465 (best epoch 2); sensor macro-F1 0.966 and 0.963. Small differences on other library versions are normal; large ones mean the split or preprocessing differs.

Reference rows: prevalence for AP (ESP validation 0.43), training mean for MAE (PE 0.517), majority class for macro-F1.

Common errors: fitting the tree on all rows of the training file; comparing a tree scored at threshold 0.5 with a network scored at a tuned threshold without saying so; overwriting the run folder (use --out per run); reporting the learning curve's standardized loss as PE error. [40 min]

Checkpoint 2: a reproducible baseline



Ask each team to rerun its command in a fresh terminal or restarted kernel while you watch; the numbers should match to the printed precision (the reference loop fixes torch, NumPy and DataLoader seeds and uses one CPU thread).

Probe the score table: are all rows scored on the same validation rows? Is the reference row there? For PE, has the team noticed that Ridge loses to the training mean? For ESP, do they know their validation contains only two entities?

A team without a working network is fine at this checkpoint if the baselines are solid; they should spend the first part of Sprint 3 on the network rather than an experiment. Lunch follows; ask teams to agree on one experiment idea before leaving. [10 min for the room]

A controlled experiment changes exactly one thing

BASELINES AND EXPERIMENTS

A controlled experiment changes exactly one thing

- Write the hypothesis and expected direction first
- Hold split, preprocessing, budget and seeds fixed
- Repeat each setting over at least three seeds
- Here, one seed at width 32 scored like width 8



Computed: reference training loop on the PE project, seeds 0–4 per width (bars mark medians); baselines fitted on the same training rows.

Neural Networks / Day 3 · Energy modelling capstone 51

[After lunch, 13:25.] Read the figure as an experiment report. Hypothesis: a wider network lowers validation MAE on the PE block. Five seeds per width, everything else fixed. Medians: width 8 0.481, width 32 0.414, width 128 0.392. Every run beats the Ridge baseline (0.546), but two runs (0.519, seed 3 at widths 8 and 32) are slightly worse than the training mean (0.517).

The seed spread is the lesson. At width 32, four seeds land between 0.407 and 0.417, but seed 3 kept its epoch-1 checkpoint and scored 0.519, like the worst width-8 run; the reference run with seed 42 scored 0.465. The spread within width 32 (0.11) is five times the median gain from 32 to 128 (0.022). A single run per setting could have supported either conclusion.

Early stopping interacts with seeds: when the validation curve dips, rises and dips again, patience decides which dip is kept. Report medians and ranges, not the best seed. In curriculum/capstone.py the validation split has its own seed (split_seed, 42 by default), so the training seed changes only the initialization and the batch order: every run is scored on the same validation rows, and comparing two settings seed by seed isolates the setting. [4 min]

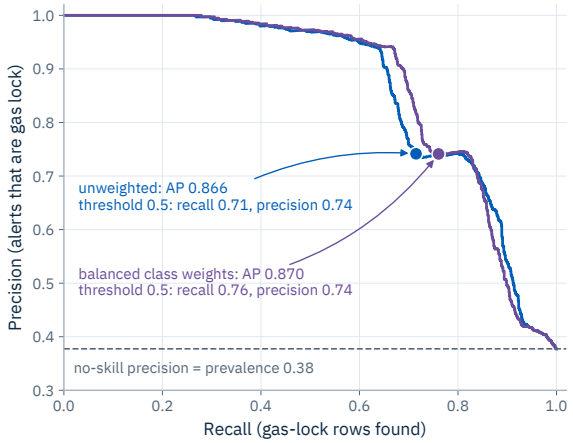
Class weights move the operating point, not the ranking

BASELINES AND EXPERIMENTS

Class weights move the operating point, not the ranking

$$L = -\frac{1}{n}\sum_i [w_1\, y_i \log p_i + w_0\,(1-y_i)\log(1-p_i)]$$

- Balanced weights set $w_c = n/(2n_c)$ for each class c
- ESP: AP 0.866 → 0.870; recall at 0.5 rises 0.71 → 0.76
- A threshold chosen on validation does a similar job



Computed from esp_train.csv: logistic regression, leave-one-entity-out predictions pooled over seven entities.

Neural Networks / Day 3 · Energy modelling capstone 52

Symbols: n is the number of training rows, n_c the number in class c, p_i the predicted probability of gas lock for row i, y_i its label. Weighting the positive class more makes the optimizer push probabilities up for positive-looking rows, so more rows cross a fixed threshold.

Numbers for the reference ESP training portion: 2,295 rows, 816 positive, 1,479 negative. Balanced weights are w_1 = 2,295 / (2 × 816) = 1.41 and w_0 = 2,295 / (2 × 1,479) = 0.78. In PyTorch the same relative weighting is BCEWithLogitsLoss(pos_weight=torch.tensor(1479 / 816)), a ratio of 1.81 (w_1 / w_0; only the overall scale of the loss differs).

On this file the ranking barely changes (average precision 0.866 versus 0.870), while the operating point at threshold 0.5 moves from recall 0.71 to 0.76 at the same precision (0.74). With 38% positives, weighting is optional; it matters more for rare events (ML Module 5). Test it as a controlled change, not as a default: it is a natural first experiment for ESP teams. [3 min]

Each project needs a matching loss and metric

BASELINES AND EXPERIMENTS

Each project needs a matching loss and metric

Project	Output	Loss	Evaluation
ESP	One logit	Binary cross-entropy with logits	PR metrics, confusion counts
PE	One numeric value	MSE or Huber	MAE, RMSE, residuals
Sensor	One logit	Binary cross-entropy with logits	Macro-F1, per-class metrics

Train on the loss, select on validation loss or metric, and report the metric in target units.

◆ Probabilities and thresholds belong to the decision step, not to the training loss.

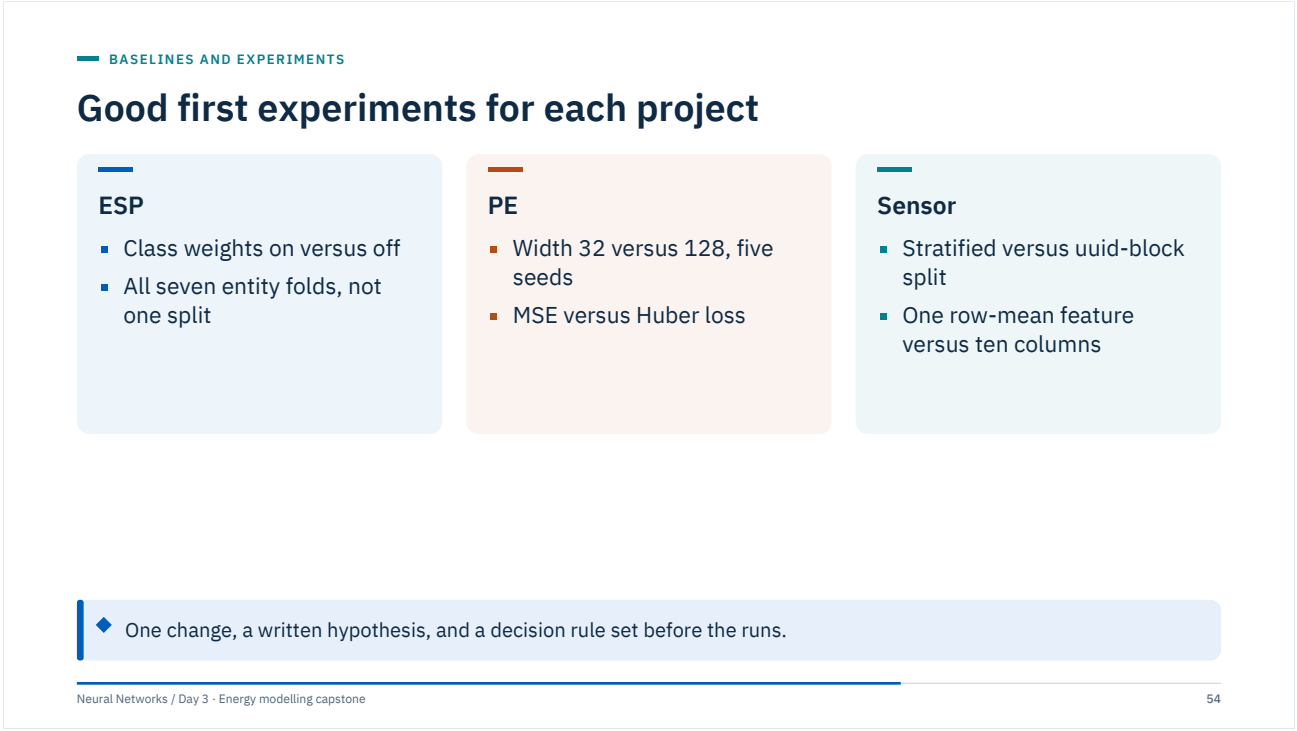
Neural Networks / Day 3 · Energy modelling capstone 53

Use the stable raw-logit loss from Day 2 (BCEWithLogitsLoss): the network outputs one unbounded number and the loss applies the sigmoid internally in a numerically safe way. For classification, probabilities and thresholds belong to prediction and decision steps, not to the training loss.

For regression, the reference trains on a standardized target with MSE; Huber loss is a reasonable controlled change if residuals show a few large misses, such as the 28-row block we will meet in error analysis. Ensure any target transform is included in the saved prediction workflow: the reference saves the target scaler and inverts it before reporting MAE and RMSE in PE units.

Ask: why macro-F1 for the sensor project rather than accuracy? Both classes matter, and macro-F1 weights them equally even though 58% of rows are target 1. [2 min]

Good first experiments for each project



ESP: class weights test whether a shifted operating point helps held-out entities; the seven-fold leave-one-entity-out run tests whether the two-entity validation was representative. Avoid per-entity normalization that uses the whole entity history: it borrows future data. A variant that normalizes each entity by statistics from its own first 24 hours, and scores only later rows, is legitimate but more advanced.

PE: width is the cheapest capacity change and the figure shows what to expect; Huber loss tests whether a few large residuals dominate training.

Sensor: the split comparison tests how much the random split flatters the model; the row-mean feature tests whether ten near-copies add anything beyond one signal.

A decision rule example: "adopt width 128 if its median validation MAE over five seeds is lower than width 32's and its worst seed is no worse than width 32's median." [3 min]

Sprint 3: run one controlled experiment

LABORATORY

40 min

Sprint 3: run one controlled experiment

- ☐ Write the hypothesis, the change and the decision rule first
- ☐ Run both settings over at least three seeds; change nothing else
- ☐ Compare the difference with the seed spread
- ☐ Decide, then write the locked configuration down

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Hypothesis, result and decision in three sentences
- Seed-by-seed table for both settings
- Locked configuration: width, lr, seed, epochs, threshold rule

Neural Networks / Day 3 · Energy modelling capstone 55

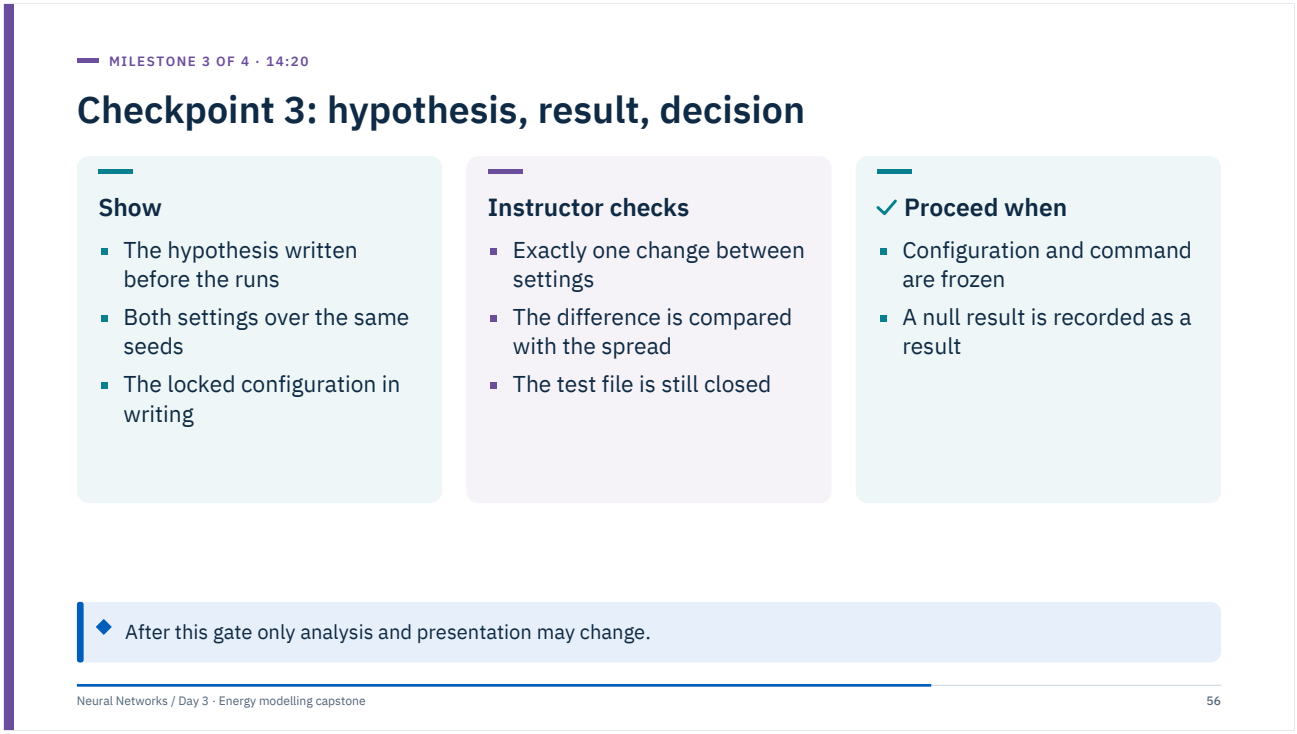
Notebook section 7. The reference run() accepts width, lr, seed, split_seed and epochs; give every run its own out directory, for example run(project, epochs=40, width=128, seed=s, out=ROOT / "output" / "capstone" / f"pe_w128_s{s}"). A loop over three to five seeds per setting takes seconds to a minute on a laptop CPU. Keep split_seed at its default so every run is scored on the same validation rows, run both settings with the same training seeds, and compare them pair by pair.

Timing: 0-5 min hypothesis and decision rule; 5-25 runs; 25-35 table and spread; 35-40 write the locked configuration (the exact arguments for the final command).

Worked answer (PE, width 32 versus 128, seeds 0-4): medians 0.414 versus 0.392, width-128 range 0.385-0.397, width-32 range 0.407-0.519. The decision rule from the previous slide is met (lower median, worst width-128 run 0.397 below the width-32 median 0.414), so adopt width 128. Lock the seed declared before the runs (for example 42) rather than the best-scoring one, and report the seed spread in the presentation.

Common errors: changing width and learning rate together; keeping the best seed and discarding the rest; using the test file to break a tie; forgetting that the threshold rule is part of the locked configuration. [40 min]

Checkpoint 3: hypothesis, result, decision



The written lock is the gate: width, learning rate, seed, epoch budget, threshold rule and the exact command, copied into the notebook's final cell. After this checkpoint the only allowed changes are to analysis and presentation.

Praise null results stated with their spread ("width made no difference beyond seed noise") as much as positive ones. Challenge claims that rest on one seed or on a difference in the third decimal.

If a team discovers a bug after locking (for example a wrong split), fixing it is allowed, but the team must say so in the presentation and rerun the whole pipeline, not just the final step. [10 min for the room]

Error analysis



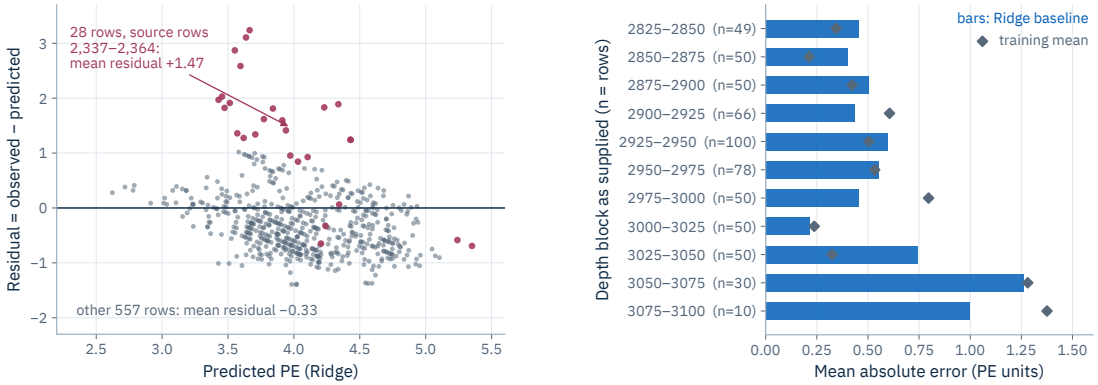
Every figure in this part comes from a simple baseline (Ridge or logistic regression) fitted on the supplied training data with a proper split: the PE source-row block, or leave-one-entity-out for ESP. The techniques transfer directly to the teams' networks in Sprint 4. The common idea: slice the aggregate score into pieces you can explain, then turn each explanation into a test. [1 min]

Residuals reveal a biased 28-row block in PE

ERROR ANALYSIS

Residuals reveal a biased 28-row block in PE

Ridge baseline fitted on source rows 0–2,336 and scored on the validation block.



Computed from Log curve - Train.csv. Left: residuals against predictions. Right: mean absolute error by 25-unit Depth block (bars: Ridge; diamonds: training mean).

Neural Networks / Day 3 · Energy modelling capstone 58

Left: residual = observed − predicted. Most validation rows scatter around a mean residual of −0.33 (the model over-predicts slightly), but 28 rows (source rows 2,337-2,364) sit far above zero with a mean residual of +1.47: observed PE averages 5.44 where the model predicts 3.97. They continue the short block that jumps in depth, seen on the audit slides.

Right: errors by depth block locate the same rows: the largest errors sit in the 3,050-3,100 blocks (MAE 1.26 and 1.00). The diamonds show that the training mean is better than Ridge in several blocks.

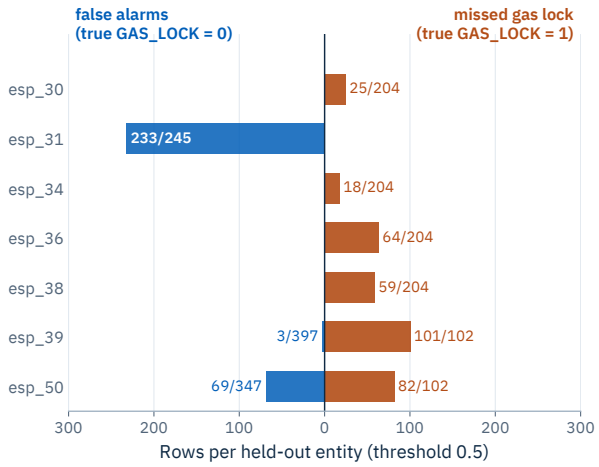
Numbers: overall MAE 0.546 for Ridge and 0.517 for the training mean; without the 28 rows 0.491 and 0.454. Two hypotheses and tests: (1) the block comes from a different well or interval: compare its input ranges with the training portion; (2) the relation is nonlinear: compare with the network's residuals on the same rows. [4 min]

Two held-out entities fail in two different ways

ERROR ANALYSIS

Two held-out entities fail in two different ways

- esp_31: every gas-lock row found, 233 of 245 normal rows flagged
- esp_39 and esp_50: most gas-lock rows missed
- Pooled, these failures average into one number
- Each failure suggests a different next test



Computed from esp_train.csv: logistic baseline, leave-one-entity-out, threshold 0.5.

Neural Networks / Day 3 · Energy modelling capstone 59

Each bar comes from a model trained on the other six entities. esp_31 is a threshold problem: its average precision is 0.989, so the ranking is almost perfect, but its probabilities are shifted upward and a fixed 0.5 threshold flags nearly every normal row. esp_39 and esp_50 are representation problems: average precision 0.356 and 0.253 against positive rates of 0.204 and 0.227, little better than no skill, and 101 of 102 and 82 of 102 gas-lock rows missed. Their signature differs: in the other five pumps PDP falls by about 190-270 during labelled periods, while in these two it rises slightly, so a model trained on the others does not recognize it.

Next tests differ accordingly: for esp_31, a per-entity or recalibrated threshold chosen without its labels; for esp_39 and esp_50, features relative to each pump's own recent baseline, or an honest statement that the model does not transfer to pumps with a different signature.

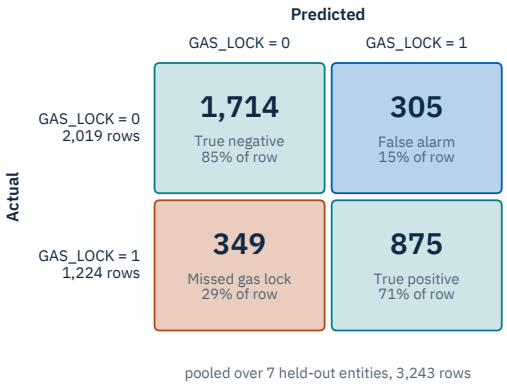
Pooled, the same model scores accuracy 0.80 and hides all of this. [4 min]

A confusion matrix separates misses from false alarms

ERROR ANALYSIS

A confusion matrix separates misses from false alarms

- Rows are actual classes; columns are predictions
- 349 of 1,224 gas-lock rows missed (29%)
- 305 of 2,019 normal rows flagged (15%)
- Decide which error costs more before moving the threshold



Computed from esp_train.csv: logistic baseline, leave-one-entity-out predictions pooled over seven entities, threshold 0.5.

Neural Networks / Day 3 · Energy modelling capstone 60

From the matrix: precision = 875 / (875 + 305) = 0.742, recall = 875 / 1,224 = 0.715, accuracy = (1,714 + 875) / 3,243 = 0.798. The row percentages are per actual class, so they do not depend on the class balance.

Connect with the entity view: 233 of the 305 false alarms come from esp_31 alone, and 183 of the 349 misses come from esp_39 (101) and esp_50 (82). A confusion matrix pooled over groups is a summary; always look for where each error type lives.

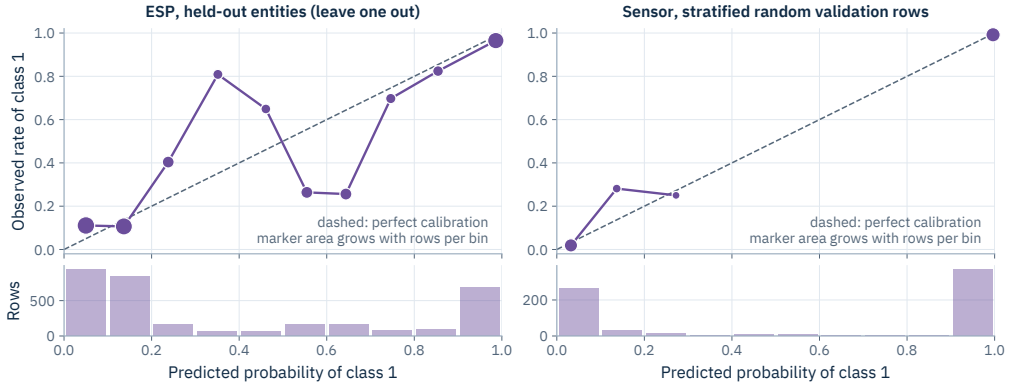
Rows are not independent events. Gas-lock periods here are 51-row blocks, so "29% of rows missed" is not "29% of gas-lock periods missed"; event-level recall (ML Module 5) would count a period as detected if enough of its rows are flagged. [3 min]

Calibration shows whether probabilities can be trusted

ERROR ANALYSIS

Calibration shows whether probabilities can be trusted

Group predictions into bins and compare the average prediction with the observed rate.



Computed: logistic baselines. Left: esp_train.csv, leave-one-entity-out (3,243 rows). Right: Sensor_train.csv, reference stratified validation rows (716). Bins with fewer than 10 rows omitted.

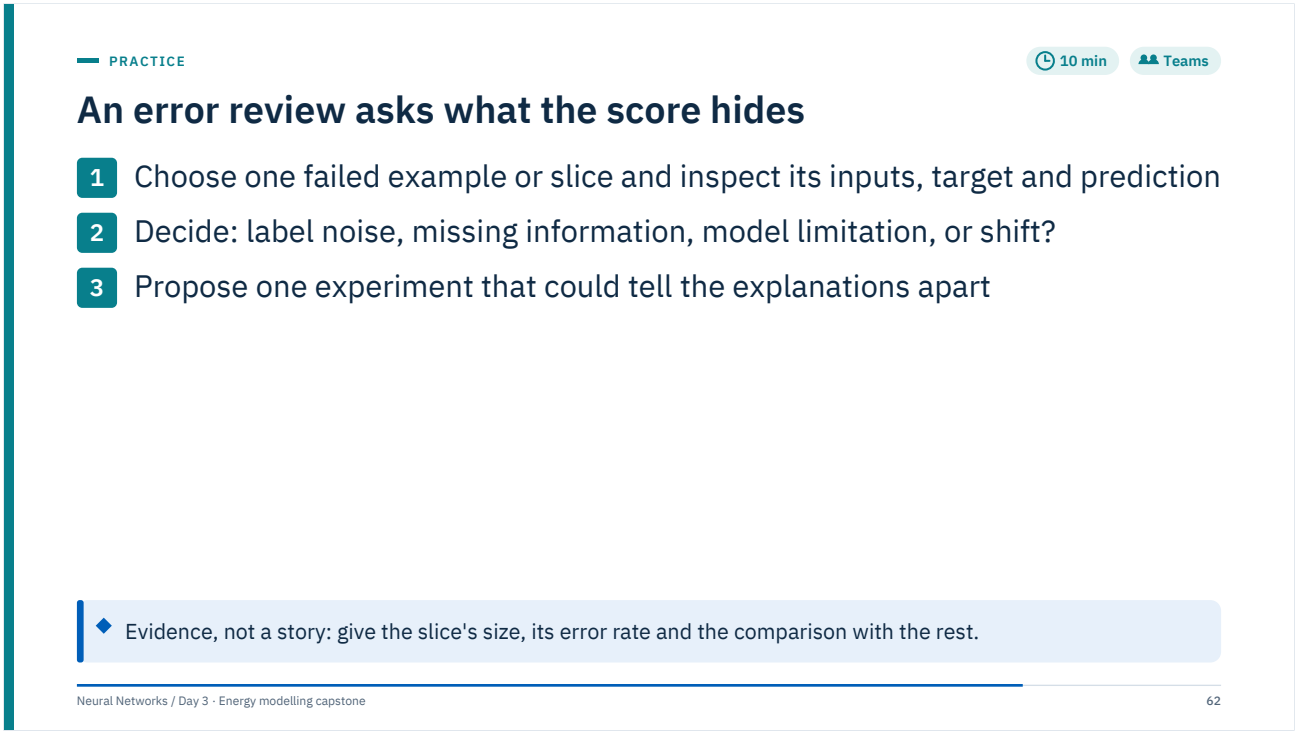
Neural Networks / Day 3 · Energy modelling capstone 61

A calibrated model's points lie on the diagonal: of the rows predicted at 0.3, about 30% are positive. Left (ESP, held-out entities): the curve is not even monotonic. Rows predicted around 0.35 are gas lock 81% of the time, rows predicted around 0.6 only 26%. Probabilities produced for unseen pumps cannot be read as frequencies, and a threshold tuned on one mix of entities may not transfer to another.

Right (sensor, random rows): most predictions sit near 0 or 1 and those bins are well calibrated; the middle bins are sparse. The random split contains near-duplicates of training rows, which makes confident predictions easy.

Remedies: report ranking metrics (average precision) alongside thresholded ones; if probabilities feed a decision, recalibrate on validation data (Platt or isotonic scaling, ML Module 5) and check calibration per group. [3 min]

An error review asks what the score hides



Allow 10 minutes, taken from the error-analysis time of Sprint 4; teams can start from validation_predictions.csv, which they have had since Sprint 2. Require evidence rather than a story about one example: a slice with its size, its error rate and the comparison with the rest.

Worked answers per project. ESP (esp_39 missed): evidence points to shift, because the labelled-period signature differs and the model's AP is near the no-skill level; experiment: rescore esp_39 with and without esp_50 (the only other pump whose PDP rises in labelled periods) in the training folds, or add features relative to each entity's own early baseline, and check whether esp_39's AP rises. PE (28-row block): shift or a stitched fragment; experiment: compare its input distributions with the training portion and rescore with the block excluded. Sensor (errors where the two classes' value ranges overlap): model limitation or missing information; experiment: compare errors under the uuid-block split, and inspect whether errors concentrate in particular uuid ranges.

Common errors: generalizing from one striking row; blaming "noise" without checking the slice size; proposing a new model rather than an experiment that separates explanations.

If inspection of final test errors inspires another model change, that new model requires a new independent assessment or a clearly stated exploratory status. [10 min, inside Sprint 4]

Sprint 4: run the final test once, then analyse errors

LABORATORY

40 min

Sprint 4: run the final test once, then analyse errors

- [ ] Section 8: run once with the locked configuration and final=True
- [ ] Confirm the uuid join or the positional row count
- [ ] Make one error-analysis figure by entity, depth block or class
- [ ] Draft slides 1–4 while the numbers are fresh

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Final score table: baseline and candidate side by side
- One error-analysis figure with a caption
- Draft slides 1–4

Neural Networks / Day 3 · Energy modelling capstone 63

Command line equivalent (ESP example with the reference settings): python3 curriculum/capstone.py esp --epochs 40 --width 32 --lr 0.001 --seed 42 --final --out output/capstone/esp_final. It trains on the designated training portion, restores the selected checkpoint, scores the supplied test file once and writes final_predictions.csv next to results.json.

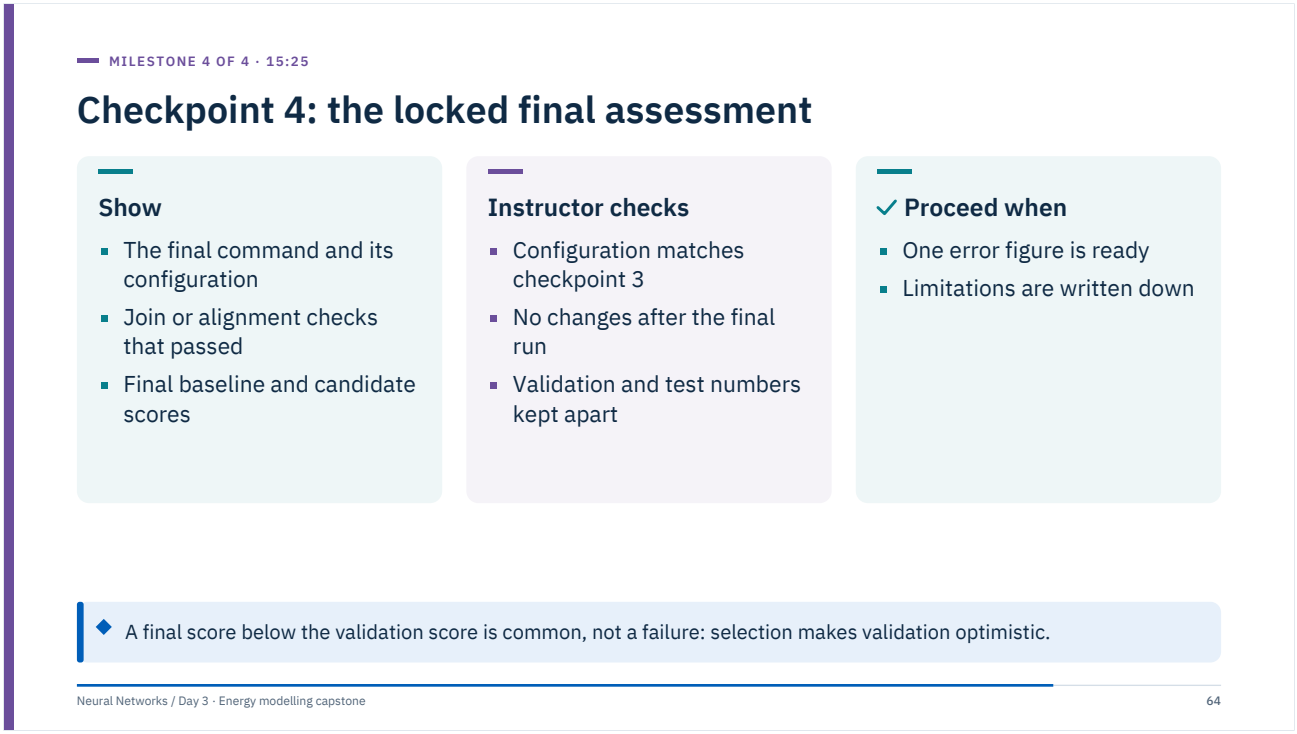
Timing: 0-5 min final run; 5-10 check the join or row count and record both baseline and candidate test scores; 10-20 the error-review exercise; 20-30 one error-analysis figure from validation_predictions.csv (entity, source row, uuid) and, for the final run, final_predictions.csv; 30-40 draft slides.

Expect test scores to differ from validation scores, often downwards: validation numbers were used for selection and are optimistic. Report both, labelled.

Instructor reference, default configuration (width 32, seed 42, 40 epochs), final run: ESP average precision 0.817 for the baseline (validation 0.968) and 0.581 for the network (0.945), with gas-lock recall of only 0.43 and 0.05 at the thresholds chosen on validation; PE MAE 0.784 and 0.812 (validation 0.546 and 0.465); sensor macro-F1 0.965 and 0.964. The ESP drop is the entity shift in action: thresholds tuned on two pumps do not transfer to three others.

Common errors: running final more than once with different settings; tuning the threshold on test labels; mixing validation and test numbers in one table without labels; for PE, reordering the test inputs before predicting. [40 min]

Checkpoint 4: the locked final assessment



Compare the final command with the configuration written at checkpoint 3; any difference must be explained. Ask to see the join assertion output (ESP and sensor) or the positional row-count check (PE).

A team whose final score is worse than its validation score has not failed; ask what the difference suggests (selection optimism, entity or block shift) and make sure the presentation says it.

If a team ran final more than once, the later runs are exploratory. They should present the first locked result as the assessment and describe the rest honestly; the rubric's evaluation criterion rewards that transparency. Break follows. [10 min for the room]

Communicating evidence



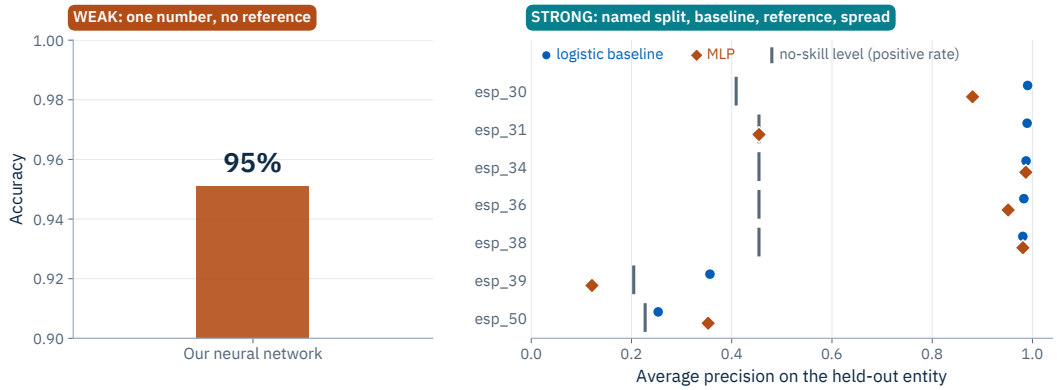
This last mini-lecture (15 minutes) turns the day's artifacts into a presentation. Everything here is about matching the strength of a claim to the strength of the evidence: name the evaluation set, show the reference, show the spread, state the limits. Ask teams to open their evidence log from the morning; every number on their slides should come from it or from results.json, not from memory. [1 min]

Strong evidence names the split, baseline and spread

COMMUNICATING EVIDENCE

Strong evidence names the split, baseline and spread

Both panels come from the same file and the same kind of network.



Computed from esp_train.csv. Left: MLP with a random 5-fold row split, pooled accuracy. Right: leave-one-entity-out average precision per entity; ticks mark each entity's positive rate.

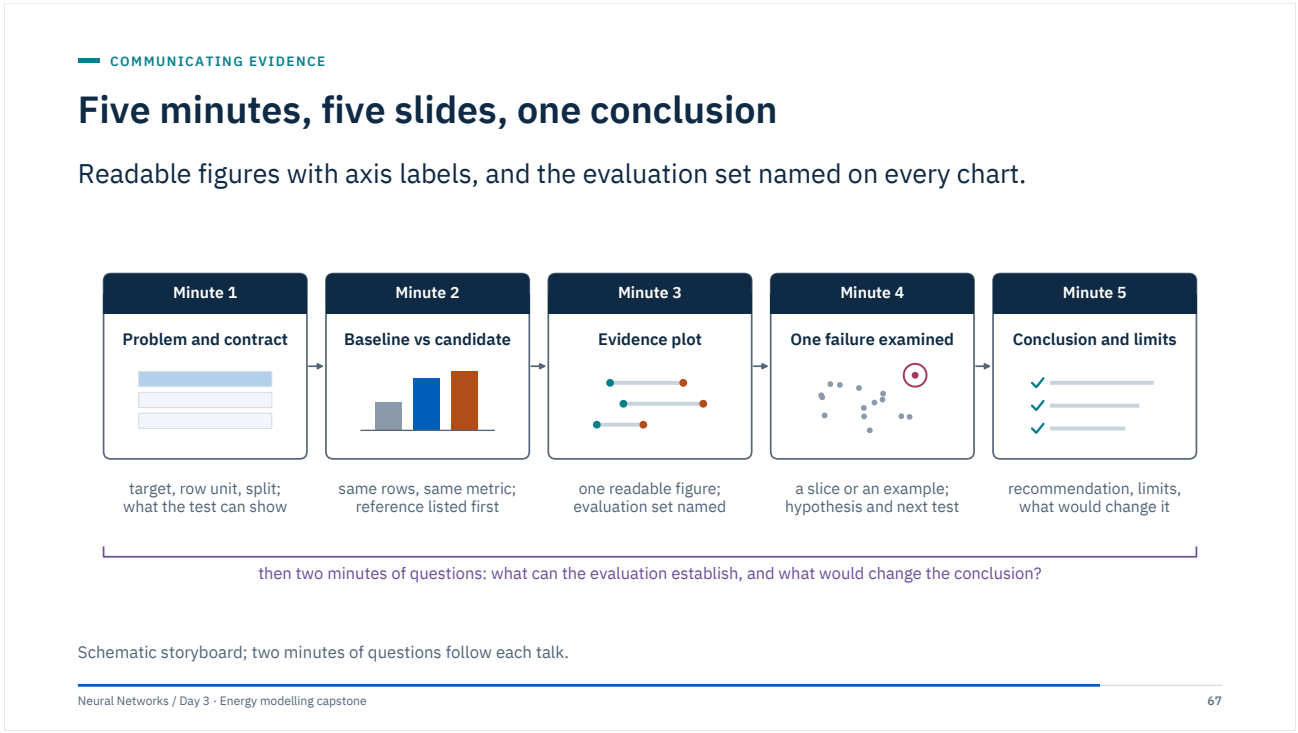
Neural Networks / Day 3 · Energy modelling capstone 66

The weak figure is typical of first drafts: one number (95%), no baseline, a truncated axis starting at 0.90, no named evaluation set, and accuracy on a target where "always no gas lock" already scores 62%. Worst of all, the random row split is the optimistic design from the opening slide.

The strong figure answers the deployment question (held-out entities), names it, shows a baseline and a no-skill reference per entity, and shows the spread. It reveals that the network is worse than logistic regression here (pooled average precision 0.611 versus 0.866), that for esp_31 the network's average precision equals the positive rate (0.454, no ranking skill), and that on esp_39 and esp_50 both models stay between 0.12 and 0.36, close to the positive rates of 0.20-0.23 (the network on esp_39 is below no skill).

Ask the room to list the differences; aim for five: split, reference, baseline, spread, metric. [4 min]

Five minutes, five slides, one conclusion

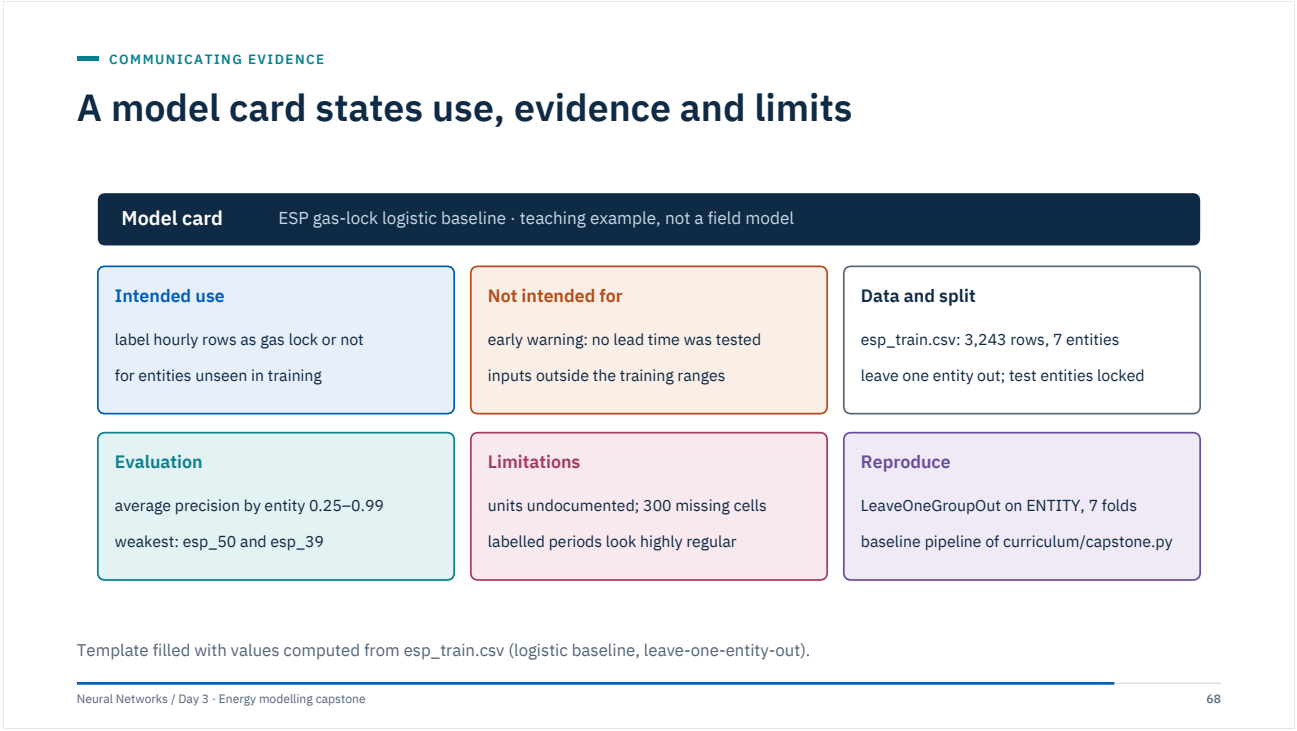


Minute 1: target, row unit, evaluation split, and what the test can and cannot show. Minute 2: baseline and neural candidate on the same rows, reference row first. Minute 3: the score table and one diagnostic plot. Minute 4: one investigated failure with a hypothesis and the test that would check it. Minute 5: conclusion, limitations and the next test.

Require readable figures with axis labels and evaluation-set names. A negative finding is a valid result. The team should answer what the model can establish, what it cannot, and why the recommendation follows.

Practical tips: one message per slide as the slide title; numbers rounded to what the seed spread supports; the limit sentence from the data contract repeated in minute 5. [3 min]

A model card states use, evidence and limits



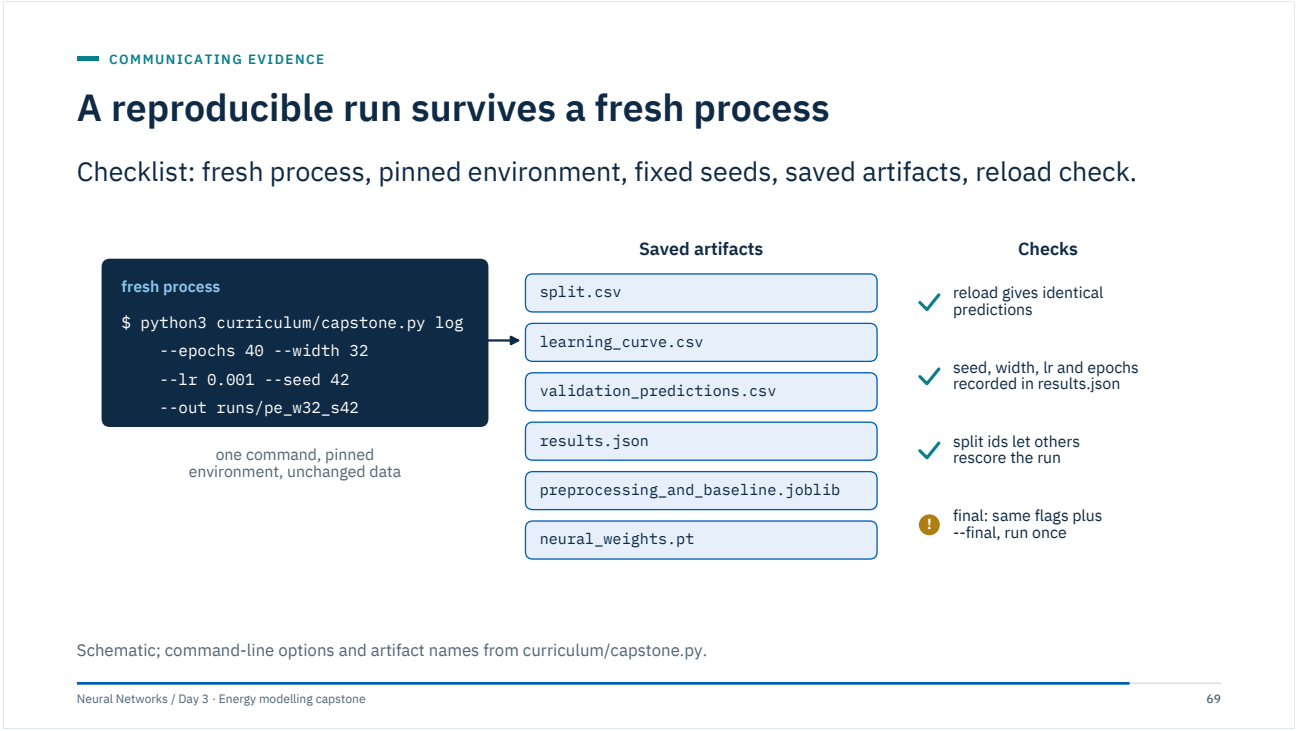
A model card (Mitchell and colleagues, "Model Cards for Model Reporting", FAT* 2019) is a short document that travels with a model: what it is for, what it is not for, the data and evaluation behind it, known limits, and how to reproduce it. ML Module 2 covers model cards in more depth; today half a page is enough and can double as slide 5.

The example is filled with real values from this file: average precision per held-out entity ranges from 0.25 (esp_50) to 0.99 (esp_30) for the logistic baseline. The "not intended for" box is the most important one: this classifier says nothing about advance warning, and nothing about inputs outside the training ranges.

Ask each team to write their card's "not intended for" box before Sprint 5; it sharpens the conclusion slide. [2 min]

Reference: https://arxiv.org/abs/1810.03993

A reproducible run survives a fresh process



Reproducibility checklist: one command from a fresh process; a pinned environment (requirements-labs.txt); fixed seeds (the reference sets torch and NumPy seeds, seeds the DataLoader generator and uses one CPU thread); saved split identifiers (split.csv); preprocessing saved with the model (preprocessing_and_baseline.joblib) and the selected weights (neural_weights.pt); configuration, metrics and library versions in results.json; unchanged data files (record file hashes if data are replaced); and a reload check that predictions are identical after loading the saved artifacts.

The last check is the one teams skip. The reference workflow asserts that eight validation predictions match to 1e-6 after reloading. Serialized Python artifacts should only be loaded from trusted sources.

GPU runs can differ slightly between runs even with seeds; on these small CPU runs the numbers should match exactly on the same machine. [2 min]

The rubric weights evaluation above accuracy

COMMUNICATING EVIDENCE

The rubric weights evaluation above accuracy

Criterion	Weight	Full-credit evidence
Problem and data contract	20%	Correct target, schema, and timing
Evaluation design	30%	Appropriate split; preprocessing fitted on training rows only
Modelling and reproducibility	20%	Baseline, candidate, saved run
Error analysis	20%	Relevant metrics and investigated failures
Communication	10%	Clear, supported conclusion and limitations

◆ Leakage or misaligned labels invalidate the affected assessment, whatever the score.

Neural Networks / Day 3 · Energy modelling capstone 70

Scores are a proposed teaching rubric, not model performance. A leakage-compromised evaluation cannot receive full marks for evaluation even with a high accuracy. Reward clear identification of missing metadata and honest limits.

A failed or inconclusive neural model can earn full marks with sound evidence. Leakage or incorrect target alignment invalidates the affected assessment.

Show the rubric before Sprint 5 so teams can check their slides against it. Evaluation design carries the most weight on purpose: the opening slide showed how much a split can move a score. Hand out a printed copy for scoring during the presentations. [2 min]

Handoff checklist: a run others can repeat

LABORATORY

15 min

Handoff checklist: a run others can repeat

- ☐ Re-run the notebook or script from a fresh process
- ☐ Include split identifiers, preprocessing, configuration and model state
- ☐ Export predictions with identifiers or original row positions
- ☐ Submit the five-slide report and one reproducible command

EVIDENCE TO SUBMIT

- Five slides
- One command and its run folder
- Predictions with uuids or row positions

Neural Networks / Day 3 · Energy modelling capstone 71

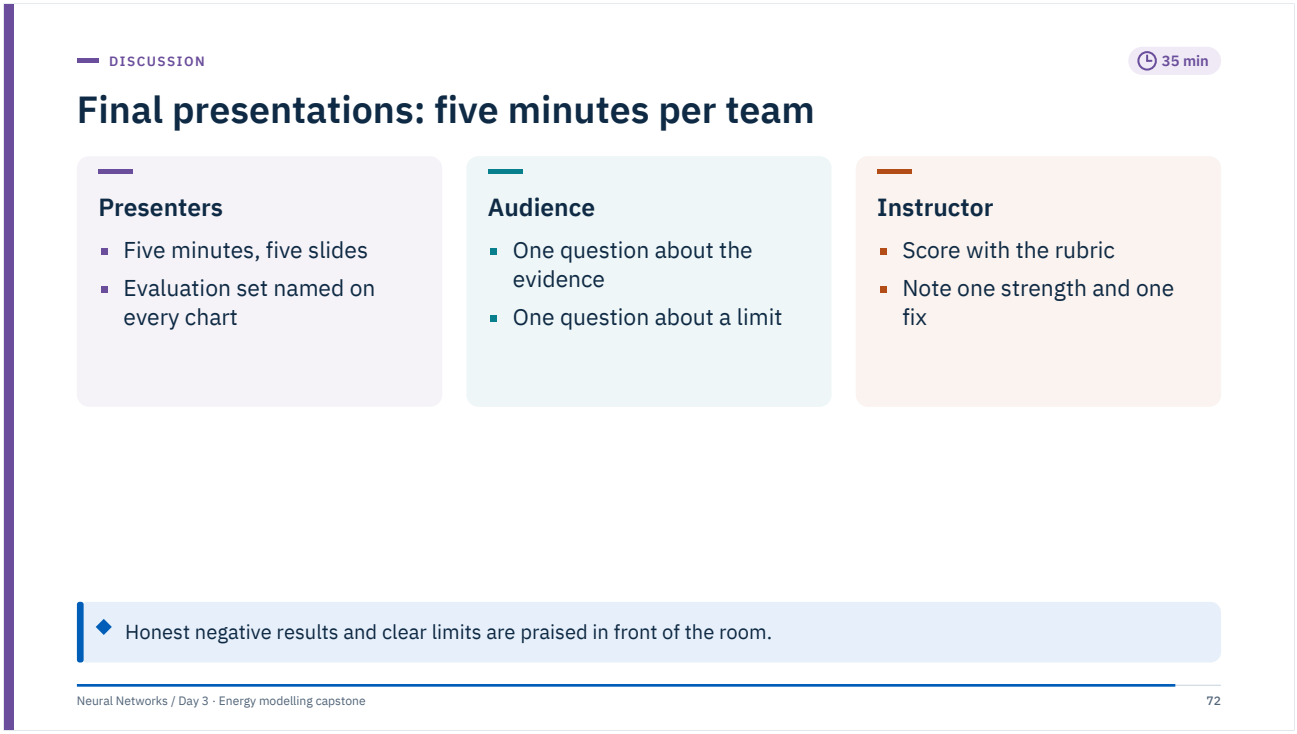
This is Sprint 5 (16:00-16:15). The capstone section of INSTRUCTOR_GUIDE.md lists the commands, data contracts and evaluation limits for each project. Keep original data files unchanged. Instructor solution files and final labels should be withheld from learners until the final evaluation stage where feasible.

Timing: 0-5 min fresh-process rerun; 5-12 finish slides; 12-15 copy the command and run folder to the shared location.

Checks to make while circulating: predictions carry uuid (ESP, sensor) or source-row position (PE); the run folder includes results.json with versions; the slides name the evaluation set on every chart.

Common errors: a notebook that only works when cells run out of order; a run folder from an earlier configuration; slides with validation and test numbers in the same unlabelled table. [15 min]

Final presentations: five minutes per team

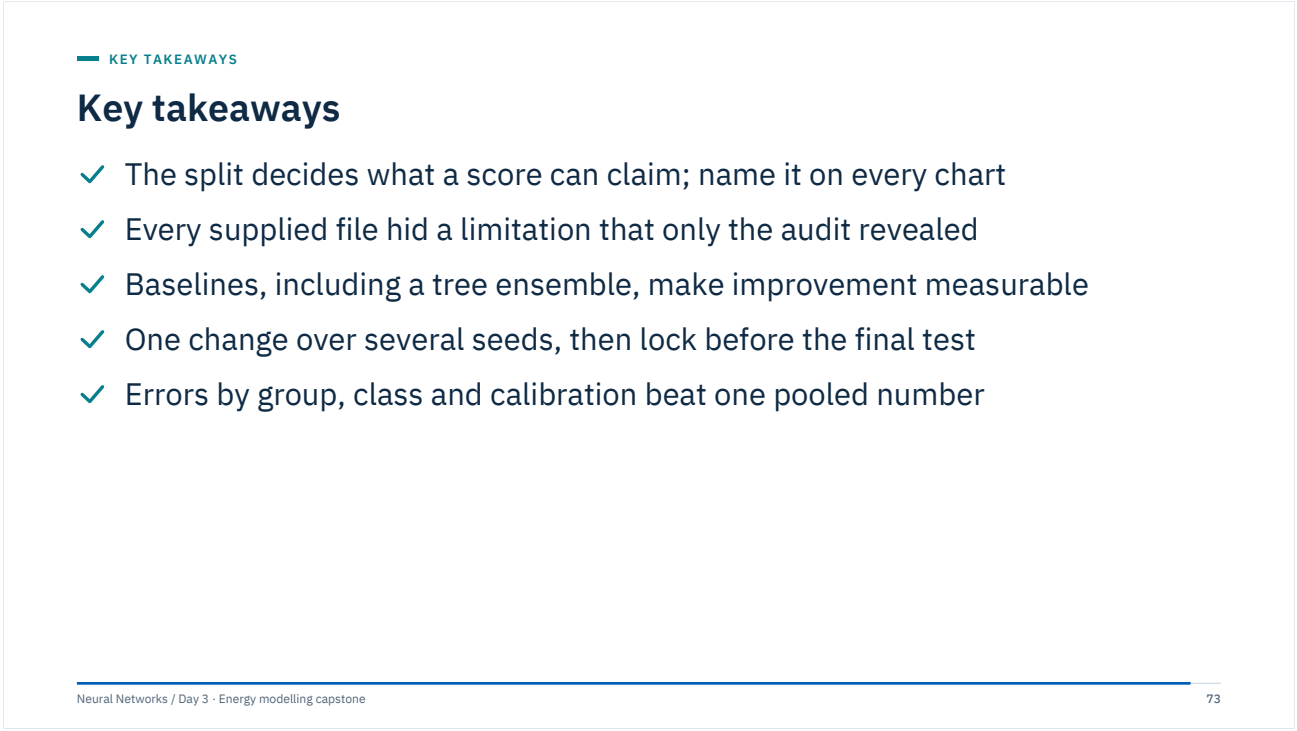


Run order: mix projects so that consecutive talks compare different kinds of evidence (for example ESP, PE, sensor, ESP, PE). Keep time strictly: a visible countdown, a one-minute warning, and a hard stop at five minutes. Two minutes of questions follow each talk.

Good audience questions: "Which rows are in your evaluation set?", "What would change your conclusion?", "How big is the seed spread compared with your improvement?", "What can your test score not tell us?"

Score with the rubric as each team talks, then give one strength and one concrete fix. Praise honest negative results and clear limits explicitly in front of the room; it sets the norm for the rest of the course. [35 min]

Key takeaways



Summarize with the day's evidence. The split: gradient boosting scored 0.96 accuracy on random ESP rows and 0.71 on held-out pumps. The audit: all-missing rows in one pump, a duplicated PE block, sensor records that repeat each other. The baselines: Ridge lost to the training mean on the PE block. The seeds: one width-32 seed looked like width 8. The errors: one pooled accuracy hid a threshold problem in one pump and a signature problem in two others.

Days 4 and 5 add architectures (convolutions and sequence models); none of them changes these rules. Transition to the exit check. [2 min]

Exit check

PRACTICE

5 min · Individual

Exit check

1. A team reports 95% accuracy for ESP from a random row split. What two things do you ask for?
2. Why is a PE validation MAE not evidence of cross-well performance?
3. A network beats logistic regression by 0.003 macro-F1 on one seed. What do you conclude?
4. Name one thing you may not do after the final test run.

Neural Networks / Day 3 · Energy modelling capstone 74

Answers.

1. The same metric under an entity-held-out split (leave-one-entity-out), and a reference plus a baseline on those rows, ideally per entity. The opening slide showed random-split accuracy falling from 0.96 to 0.71 for gradient boosting when entities are held out; also ask why accuracy rather than precision and recall for gas-lock rows. 2. There is no well identifier. The validation block is a contiguous set of source rows whose relation to wells is unknown, so it can show performance on a later block of this file, not on new wells. 3. Nothing yet. The difference is far smaller than typical seed-to-seed spread; repeat both over several seeds (and ideally a uuid-block split) and compare the difference with the spread. 4. Any of: change the configuration and rerun as if the result were still a final assessment; choose the threshold on test labels; drop test rows that failed the join; report only the better of several final runs.

Collect answers on paper or in the chat; question 3 is the one most often missed. [5 min]

References and further study

FURTHER STUDY

References and further study

- Grinsztajn, Oyallon and Varoquaux (2022). *Why do tree-based models still outperform deep learning on typical tabular data?* NeurIPS Datasets and Benchmarks Track.
- Mitchell and colleagues (2019). *Model Cards for Model Reporting.* Conference on Fairness, Accountability, and Transparency (FAT*).
- scikit-learn User Guide: cross-validation for grouped data, probability calibration, common pitfalls.
- PyTorch documentation: BCEWithLogitsLoss (pos_weight), saving and loading models.
- Course files: curriculum/capstone.py and the capstone section of INSTRUCTOR_GUIDE.md.

The two papers are the primary sources behind the tabular-data and model-card slides. The scikit-learn pages cover GroupKFold, GroupShuffleSplit and LeaveOneGroupOut, CalibratedClassifierCV and reliability diagrams, and the preprocessing-leakage pitfalls that the capstone reference workflow is built to avoid. The PyTorch page documents the raw-logit loss and its pos_weight argument.

For follow-up work, suggest that teams rerun their experiment with a stronger split (all seven ESP entity folds, or the uuid-block split for sensors) and compare with today's result. [1 min]

Reference: https://arxiv.org/abs/2207.08815 ; https://arxiv.org/abs/1810.03993 ; https://scikit-learn.org/stable/modules/cross_validation.html ; https://scikit-learn.org/stable/modules/calibration.html ; https://scikit-learn.org/stable/common_pitfalls.html ; https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html